

Assignment Data Science for Non-Life Insurance

Sofia Allosia - Student Number: r1125708
Luisa Scaroni - Student Number: r1085999
Garvit Malik - Student Number: r0870924

First, we load the required libraries:

```
library(tidyverse)
library(skimr)
library(readxl)
library(corrplot)
library(psych)
```

Next, we import the data:

```
data=read_excel('insuranceportfolio.xlsx')
```

Exploratory Data Analysis

We start by defining the key variables used in our analysis:

- **Frequency** (`n_medical_services`): the *total number of medical services* utilized under the insured policy during the current year.

- **Severity** (`cost_claims_year`): the *average cost per medical event*. It is derived by dividing the total cost of claims by the number of medical services noting that the `n_medical_services` has to be different from 0.

$$\text{Severity} = \text{cost_claims_year} / \text{n_medical_services}$$

- **Covariates**: the independent variables used for modeling, it includes: *insured details* (`age`, `gender`), *policy details* (`type_policy`, `type_product`, `premium`, `distribution_channel`, `reimbursement`) and *geographical data* (`C_H`, `C_GI` and `C_C`).

We start by converting to numbers the qualitative data with `as.factor`:

```
colnames(data)
```

```
## [1] "ID"                "ID_policy"         "ID_insured"
## [4] "period"            "date_effect_insured" "date_lapse_insured"
## [7] "date_effect_policy" "date_lapse_policy"  "year_effect_insured"
## [10] "year_lapse_insured" "year_effect_policy" "year_lapse_policy"
## [13] "exposure_time"     "lapse"             "seniority_insured"
## [16] "seniority_policy"  "type_policy"        "type_policy_dg"
## [19] "type_product"      "reimbursement"     "new_business"
## [22] "distribution_channel" "gender"             "age"
```

```
## [25] "premium"          "cost_claims_year"    "n_medical_services"
## [28] "n_insured_pc"      "n_insured_mun"       "n_insured_prov"
## [31] "IICIMUN"          "IICIPROV"            "C_H"
## [34] "C_GI"              "C_II"                "C_IE_P"
## [37] "C_IE_S"            "C_IE_T"              "C_GE_P"
## [40] "C_GE_S"            "C_GE_T"              "C_C"

dataclean=data %>%
  mutate(
    gender=as.factor(gender),
    type_policy=as.factor(type_policy),
    type_product=as.factor(type_product),
    type_policy_dg=as.factor(type_policy_dg),
    reimbursement=as.factor(reimbursement),
    new_business=as.factor(new_business),
    distribution_channel=as.factor(distribution_channel),
    lapse=as.factor(lapse),
    C_H=as.factor(C_H),
    C_C=as.factor(C_C),
    severity=ifelse(n_medical_services>0, cost_claims_year/n_medical_services, NA)
  )

dataclaims=dataclean[which(dataclean$n_medical_services>0),]
```

Now all the data are **numeric**, so we will be able to conduct our analysis.

```
summary(dataclean)
```

```
##      ID      ID_policy      ID_insured      period
## Length:228711   Min.   :    1   Min.   :  1.00   Min.   :2017
## Class :character 1st Qu.: 9800   1st Qu.:  1.00   1st Qu.:2017
## Mode  :character Median :20502   Median :  2.00   Median :2018
##              Mean  :20629   Mean   : 13.94   Mean   :2018
##              3rd Qu.:31306   3rd Qu.:  3.00   3rd Qu.:2019
##              Max.   :45162   Max.   :1886.00   Max.   :2019
##
## date_effect_insured      date_lapse_insured
## Min.   :1933-06-01 00:00:00   Min.   :2017-01-01 00:00:00
## 1st Qu.:1996-06-01 00:00:00   1st Qu.:2018-01-01 00:00:00
## Median :2009-03-01 00:00:00   Median :2018-12-31 00:00:00
## Mean   :2005-07-29 05:35:27   Mean   :2018-11-13 20:03:47
## 3rd Qu.:2016-07-01 00:00:00   3rd Qu.:2019-08-01 00:00:00
## Max.   :2019-12-15 00:00:00   Max.   :2019-12-31 00:00:00
##              NA's   :167181
## date_effect_policy      date_lapse_policy
## Min.   :1932-03-01 00:00:00   Min.   :2017-01-01 00:00:00
## 1st Qu.:1995-05-01 00:00:00   1st Qu.:2018-01-01 00:00:00
## Median :2008-07-01 00:00:00   Median :2018-12-31 00:00:00
## Mean   :2004-08-23 16:53:14   Mean   :2018-11-19 00:56:59
## 3rd Qu.:2016-05-01 00:00:00   3rd Qu.:2019-08-01 00:00:00
## Max.   :2019-12-15 00:00:00   Max.   :2019-12-31 00:00:00
##              NA's   :174568
```

```

## year_effect_insured year_lapse_insured year_effect_policy year_lapse_policy
## Min. :1933 Min. :2017 Min. :1932 Min. :2017
## 1st Qu.:1996 1st Qu.:2018 1st Qu.:1995 1st Qu.:2018
## Median :2009 Median :2018 Median :2008 Median :2018
## Mean :2005 Mean :2018 Mean :2004 Mean :2018
## 3rd Qu.:2016 3rd Qu.:2019 3rd Qu.:2016 3rd Qu.:2019
## Max. :2019 Max. :2019 Max. :2019 Max. :2019
## NA's :167181 NA's :174568
## exposure_time lapse seniority_insured seniority_policy type_policy
## Min. :0.0000 1: 16348 Min. : 0.00 Min. : 0.0 C: 83240
## 1st Qu.:1.0000 2:187271 1st Qu.: 2.00 1st Qu.: 2.0 I:145471
## Median :1.0000 3: 25092 Median : 9.00 Median :10.0
## Mean :0.9354 Mean :12.78 Mean :13.7
## 3rd Qu.:1.0000 3rd Qu.:22.00 3rd Qu.:23.0
## Max. :1.0000 Max. :86.00 Max. :87.0
##
## type_policy_dg type_product reimbursement new_business distribution_channel
## C1: 25724 D: 9866 No :186818 No :197824 A: 82644
## C2: 3905 I: 3017 Yes: 41893 Yes: 30887 D: 2039
## C3: 49775 P: 40413 I:144028
## C4: 3836 S:175415
## I :115210
## SE: 30261
##
## gender age premium cost_claims_year
## F:123908 Min. : 0.00 Min. : 33.33 Min. : 0.0
## M:104803 1st Qu.:29.00 1st Qu.: 539.58 1st Qu.: 0.0
## Median :45.00 Median : 752.71 Median : 185.6
## Mean :44.41 Mean : 852.01 Mean : 587.7
## 3rd Qu.:63.00 3rd Qu.: 1024.14 3rd Qu.: 542.4
## Max. :99.00 Max. :22318.05 Max. :87000.9
##
## n_medical_services n_insured_pc n_insured_mun n_insured_prov
## Min. : 0.00 Min. : 1.0 Min. : 1 Min. : 4
## 1st Qu.: 0.00 1st Qu.: 43.0 1st Qu.: 100 1st Qu.: 2872
## Median : 6.00 Median : 110.0 Median : 994 Median : 7467
## Mean : 16.81 Mean : 282.1 Mean : 5280 Mean :13153
## 3rd Qu.: 23.00 3rd Qu.: 371.0 3rd Qu.: 4950 3rd Qu.:28298
## Max. :3094.00 Max. :2518.0 Max. :19315 Max. :29465
##
## IICIMUN IICIPROV C_H C_GI
## Min. : 0.0061 Min. : 0.0075 H1 : 2569 Min. : 1.0
## 1st Qu.: 1.6483 1st Qu.: 1.2854 H2 : 22252 1st Qu.: 74.0
## Median : 5.6034 Median : 4.9199 H3 : 27990 Median : 92.0
## Mean : 6.8977 Mean : 4.2475 H4 : 17194 Mean : 81.5
## 3rd Qu.: 11.1032 3rd Qu.: 5.3070 H5 : 18338 3rd Qu.: 98.0
## Max. :263.1579 Max. :10.4859 H6 :129482 Max. :100.0
## NA's :10886 NA's :5701 NA's : 10886 NA's :13213
## C_II C_IE_P C_IE_S C_IE_T
## Min. : 1.00 Min. : 1.00 Min. : 1.00 Min. : 1.00
## 1st Qu.: 25.00 1st Qu.: 25.00 1st Qu.: 25.00 1st Qu.: 25.00
## Median : 50.00 Median : 50.00 Median : 50.00 Median : 50.00
## Mean : 50.49 Mean : 50.48 Mean : 50.48 Mean : 50.49
## 3rd Qu.: 75.00 3rd Qu.: 75.00 3rd Qu.: 75.00 3rd Qu.: 75.00

```

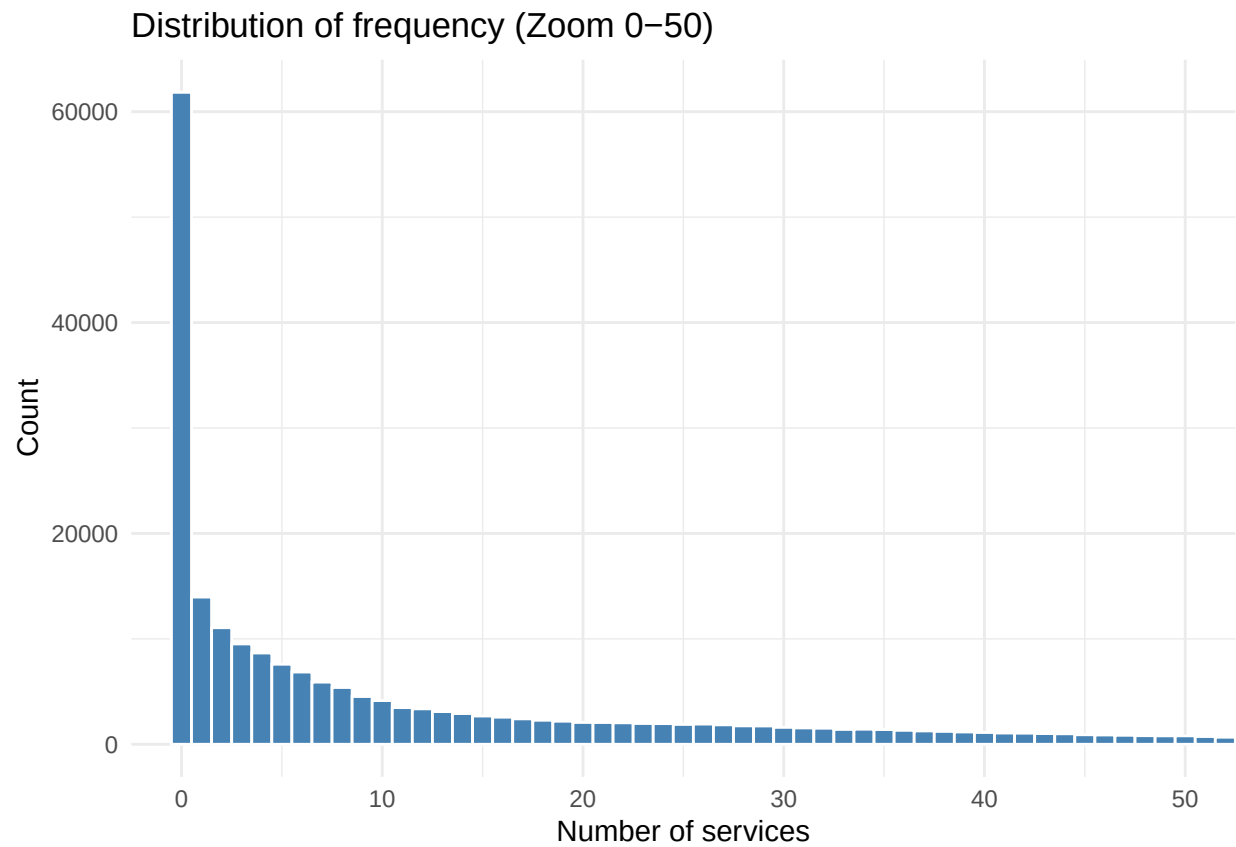
```
## Max. :100.00 Max. :100.00 Max. :100.00 Max. :100.00
## NA's :13213 NA's :13213 NA's :13213 NA's :13213
## C_GE_P C_GE_S C_GE_T C_C
## Min. : 1.00 Min. : 1.00 Min. : 1.00 C1 : 25041
## 1st Qu.: 7.00 1st Qu.: 3.00 1st Qu.: 77.00 C2 : 5926
## Median : 14.00 Median : 14.00 Median : 93.00 C3 :102994
## Mean : 20.04 Mean : 26.66 Mean : 84.14 C4 : 24571
## 3rd Qu.: 29.00 3rd Qu.: 46.00 3rd Qu.: 99.00 C5 : 6591
## Max. :100.00 Max. :100.00 Max. :100.00 C6 : 53032
## NA's :13213 NA's :13213 NA's :13213 NA's: 10556
## severity
## Min. : 0.255
## 1st Qu.: 18.200
## Median : 30.017
## Mean : 43.445
## 3rd Qu.: 47.676
## Max. :16417.550
## NA's :61834
```

A significant number of values is missing (NAs) in several geographical indices and cluster variables (IICIMUN, C_GI and C_C). Before training the predictive model it is necessary to handle these *missing values*.

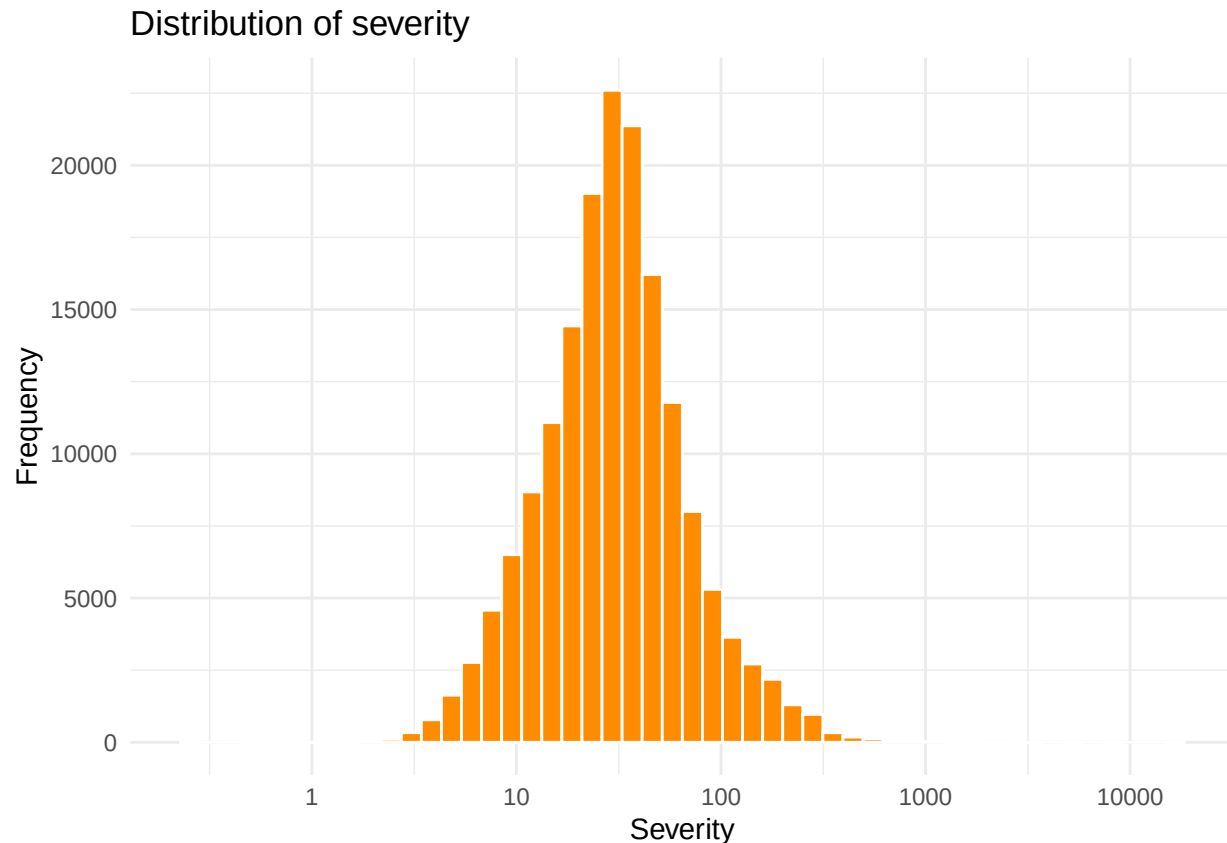
Then we note that the portfolio is concentrated in C3 area, followed by C6 and C1. This implies a focus in some specific Spanish regions.

Now we study the severity variable, it has a mean of 43.3, while having a maximum of 16 417.55. This huge difference implies a **heavily right-skewed** distribution and the presence of **severe large claims**. The frequency has a mean of 16.8 and a maximum of 3 094, indicating *overdispersion*. Conversely, the age of the insured population is well-distributed. It ranges from 0 to 99 years old, having an average of 44.4 years.

```
ggplot(dataclean, aes(x=n_medical_services)) +
  geom_histogram(binwidth=1, fill='steelblue', color='white') +
  theme_minimal() +
  coord_cartesian(xlim=c(0, 50))+
  labs(title = 'Distribution of frequency (Zoom 0-50)', x='Number of services', y='Count')
```

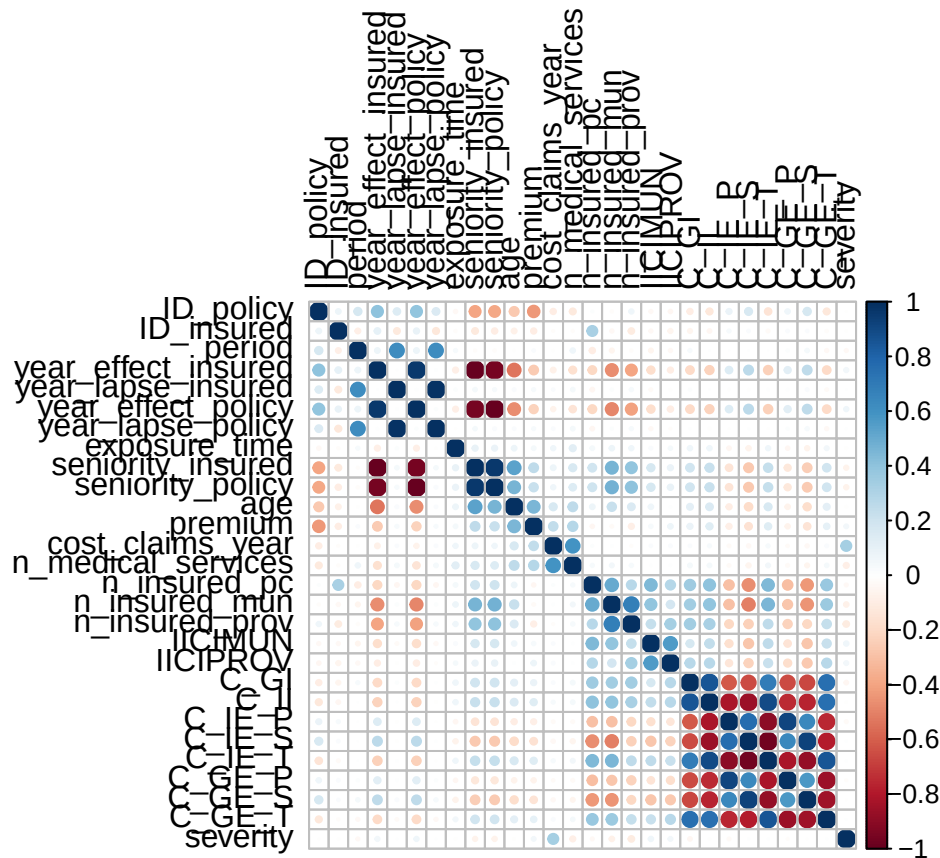



```
ggplot(dataclaims, aes(x=severity))+  
  geom_histogram(bins=50, fill='darkorange', color='white')+scale_x_log10()+theme_minimal()+labs
```



The first plot shows the **distribution of the number of medical services**. As expected for health insurance, the data are right skewed with a massive peak at zero, implying that the *majority of insured do not file any claims during the year*. Then the long right tail confirms **overdispersion** (variance>mean), this suggest that a negative Binomial model will be more appropriate than a standard Poisson. The second plot shows the distribution of claim severity. We applied a logarithmic scale to the x-axis (`scale_x_log10()`) getting a symmetric bell curve. This indicates that the original unscaled severity data is strongly right-skewed. This is common for insurance payouts, indeed most claims are of moderate value but a few extreme values pull the average up. As a consequence, this suggests that the most suitable choices for modeling severity are Log-Normal or Gamma distributions.

```
datanumeric=ataclean[,sapply(ataclean, is.numeric)]
correlation=cor(datanumeric, use='pairwise.complete.obs')
corrplot(correlation, tl.col='black')
```

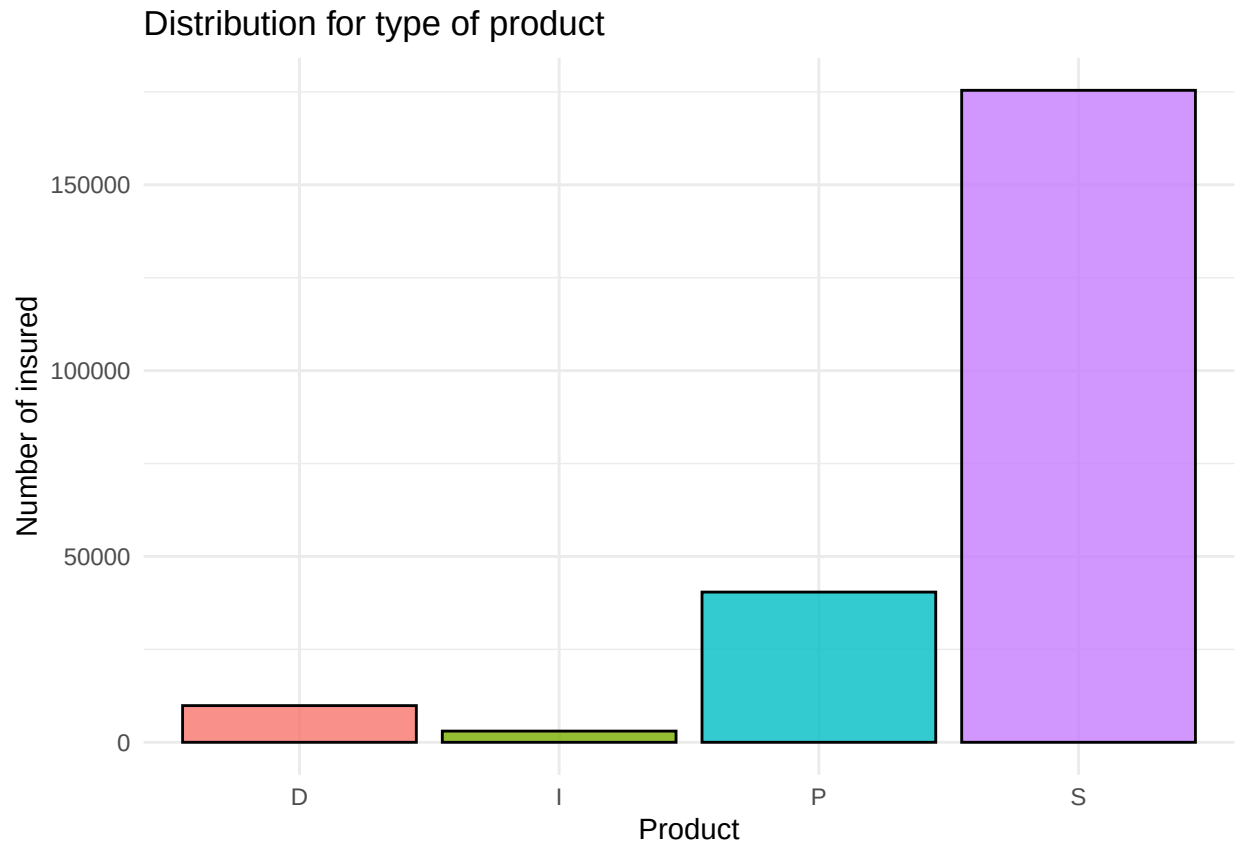


From the correlation plot we can see a **low multicollinearity**, indeed the majority of the matrix consists of small light colored dots, implying that our *features are not correlated* with each other. This also means that each variable brings **independent** information to the model. There is a strong positive correlation between temporal variables (such as `seniority_insured` and `seniority_policy`). While the effect years and seniority of the variables have a strong negative correlation. Then the geographical variables show intense positive and negative correlations. It's important to note that strong negative correlations are typical in **compositional data**. so if the percentage of one demographic increases, the percentage of another category has to decrease. In conclusion, our severity and frequency do not show strong correlations with individual numerical covariates. This confirms our idea to use GLMs or more advanced techniques.

Through the histograms of the relevant variables we were able to study the **demographic and financial profile** of our portfolio. Firstly, the “age” distribution is widespread with a peak in the 40-50 age bracket. Then, the premium is heavily right-skewed with the majority of the policy holders in the lower premium ranges. Since the `exposure_time` is dominated by a spike at 1, this means that the majority of policyholder were active for the entire observed year. The `seniority_insured` and `seniority_policy` are right-skewed, showing an influx of newer customers and a long tail of loyal longer term policyholders.

We now use ggplot in order to study the distribution based on the type of product:

```
ggplot(dataclean %>% drop_na(type_product), aes(x=type_product, fill=type_product))+
  geom_bar(color='black', alpha=0.8)+
  theme_minimal()+
  labs(title='Distribution for type of product', x='Product', y='Number of insured')+
  theme(legend.position='none')
```



The bar chart shows the distribution of the insured population across different products. We can see how it is highly **unbalanced**: 'S' dominates the business with 150 000 policies. While the other products do not exceed the 50 000 policies. The class imbalance implies that our predictive models will be trained mainly on the characteristics and behaviors of product 'S'.

We study the mean and variance of the events:

```
meanfreq=mean(dataclean$n_medical_services, na.rm=TRUE)
varfreq=var(dataclean$n_medical_services, na.rm=TRUE)
print(meanfreq)
```

```
## [1] 16.81229
```

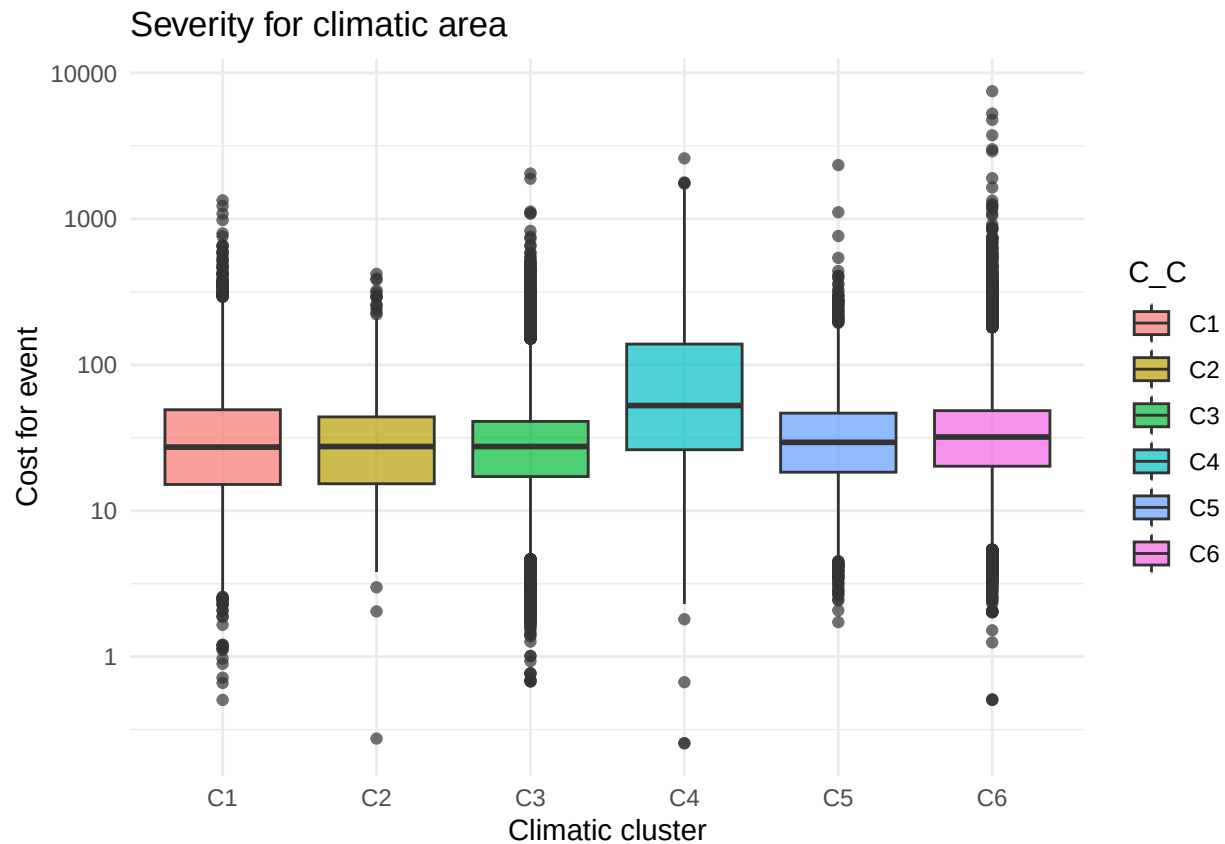
```
print(varfreq)
```

```
## [1] 796.8725
```

The variance (796.8725) is larger than the mean (16.81229). This implies *overdispersion within our claim counts*. Since standard Poisson relies on the assumption of equidispersion (mean=variance) applying it would result in underestimated standard errors. As a consequence the **negative Binomial distribution** is the most appropriate choice.

We can now study the geographic:

```
ggplot(dataclaims %>% drop_na(C_C), aes(x=C_C, y=severity, fill=C_C))+geom_boxplot(alpha=0.7)+scale_y_l
```

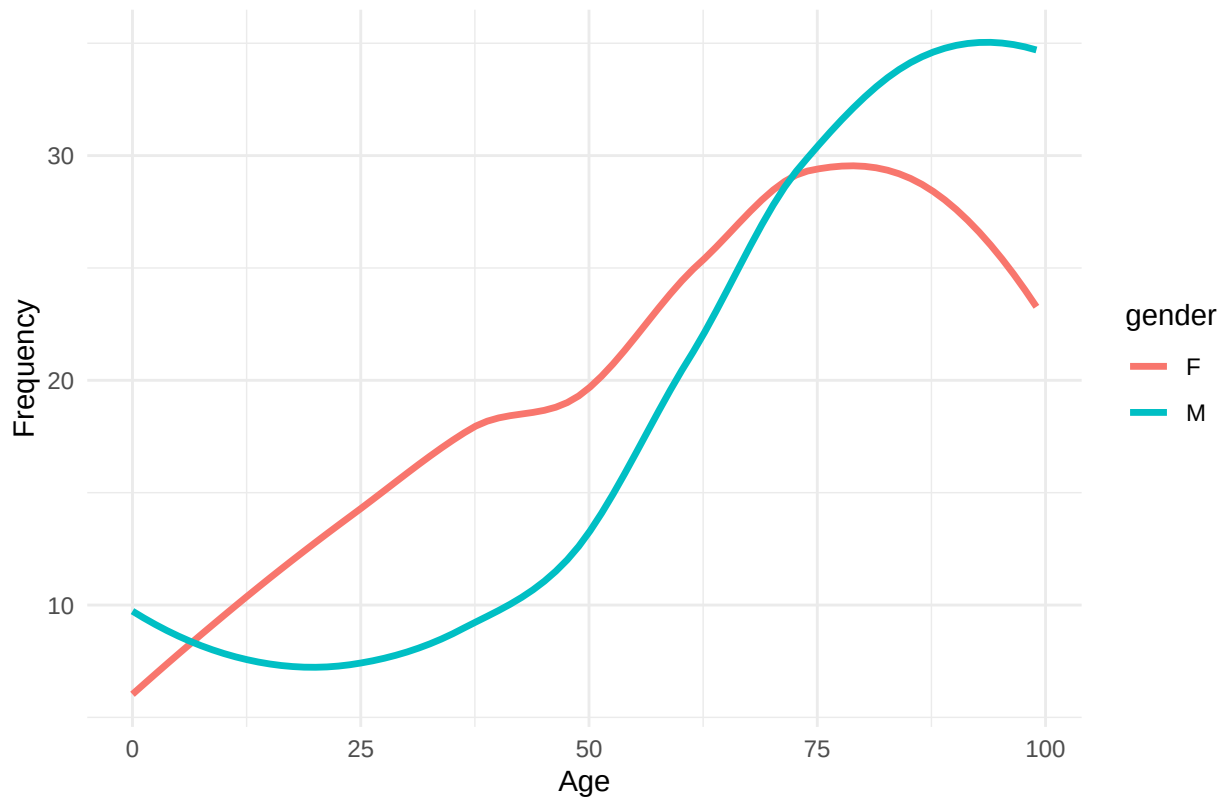


We displayed the boxplot on a *logarithmic scale*, then we were able to study the distribution of claim severity across the six climatic areas. Firstly, while C1, C2, C3, C5 and C6 show a similar median claim cost, cluster C4 stands out significantly. It has an higher median severity and an upward-shifted interquartile range. This implies that the events that occur in this region are more expensive. Then we can see a lot of black dots above the upper whiskers representing severe outliers. All of this shows that geographical location is highly relevant when building the severity predictive model.

We now try to do a **multivariate analysis** to see how covariates are connected/correlated:

```
dataagegender=ataclean %>%
  group_by(age, gender) %>%
  summarise(
    exposure = sum(exposure_time, na.rm = TRUE),
    claims = sum(n_medical_services, na.rm = TRUE),
    frequency = claims / exposure,
    .groups = "drop"
  ) %>%
  filter(exposure > 10) %>% drop_na(gender)
ggplot(dataagegender, aes(x=age, y=frequency, color=gender))+ geom_smooth(method='loess', se=FALSE, s
  theme_minimal()+labs(title='Frequency of events: age vs gender', x='Age', y='Frequency')
```

Frequency of events: age vs gender



We can observe a *non-linear pattern* crossing in two points, we can see three distinct phase:

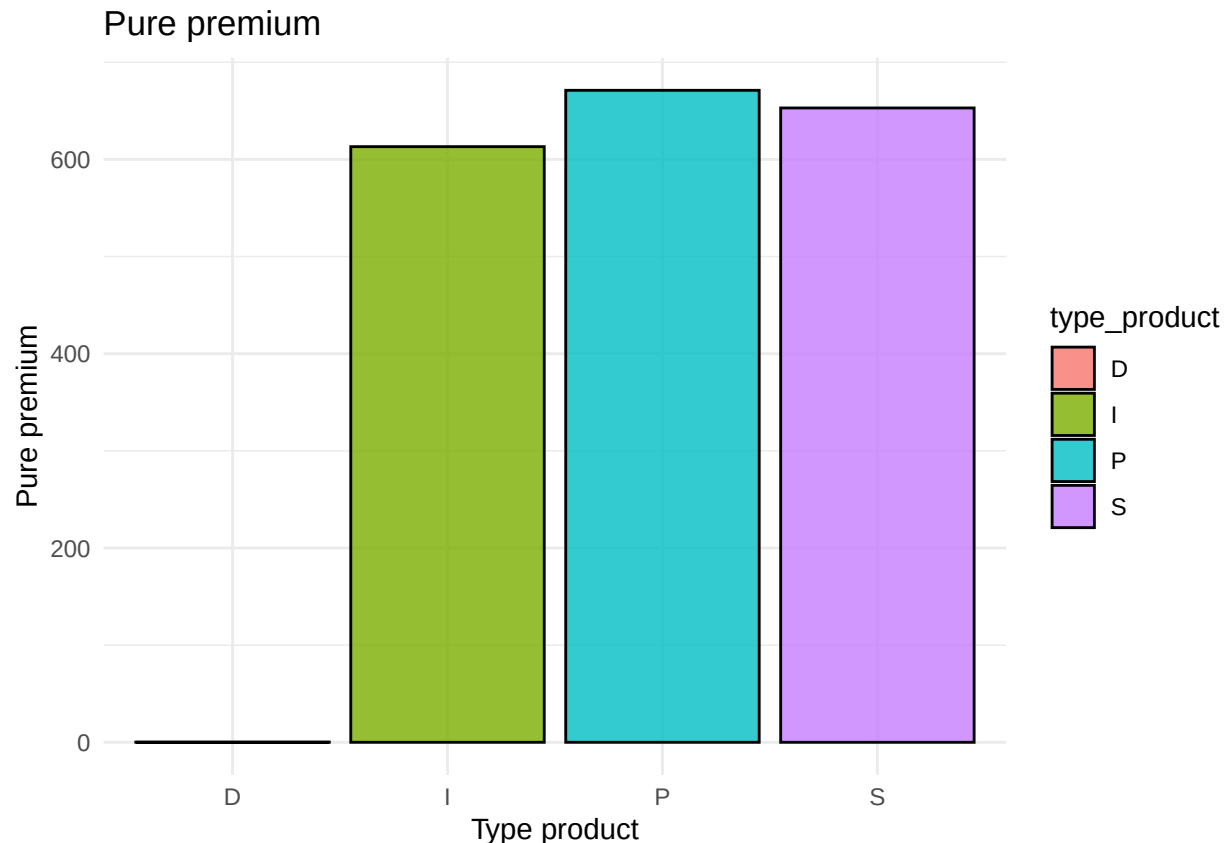
-**Young and Middle Adulthood** (Age 15-65): females show an higher claim frequency than males. This is an expected pattern driven by maternity-related medical services, preventive care and higher proactive approach in women.

-**The Crossover Point**(Age 65-70): as the policyholders enter their late 60s, the two curves intersect. The female frequency starts to plateau and decline, while the male frequency continues an upward trajectory.

-**Seniority Years**(Age 70+): in the older demographic, males have an higher frequency of medical events compared to females. This is connected to chronic conditions, cardiovascular diseases and other age related deterioration in older men.

We now calculate and visualize the **expected pure premium** across different product types:

```
datapurepremium=dataclean %>%
  drop_na(type_product) %>%
  group_by(type_product) %>%
  summarise(
    exposure = sum(exposure_time, na.rm = TRUE),
    totcost = sum(cost_claims_year, na.rm = TRUE),
    pureprem = totcost / exposure,
    .groups = "drop"
  )
ggplot(datapurepremium, aes(x=type_product, y=pureprem, fill=type_product))+ geom_col(color='black',
theme_minimal()+labs(title='Pure premium', x='Type product', y='Pure premium')
```



The bar chart illustrates the observed pure premium for each product type. It represents the baseline expected cost per unit of exposure that the insurance company needs to cover. For the products 'I', 'P' and 'S' we get similar pure premiums, all clustering in the 600-700 range. Noting that 'P' shows the highest expected cost, implying that it is the most expensive risk profile. In contrast, product 'D' displays a premium near zero implying a very low number of insured. This suggests that this portfolio experienced almost no severe claims during the observation period

We study the large claims:

```
severitypercentile=quantile(dataclaims$severity, probs=c(0.5, 0.75, 0.9, 0.95, 0.99 ), na.rm=TRUE)
print(severitypercentile)
```

```
##          50%          75%          90%          95%          99%
## 30.01714  47.67604  81.77756 124.23000 258.09125
```

The data shows that 50% of the claims cost around 30 and 95% of all medical events cost less than 125. We can see that even at the 99th percentile the severity is approximately 258. However, the maximum observed severity is over 16 400. This confirms that the insurer's financial risk is influenced by less than 1% of extremely expensive events, this is referred as Large Claims. Standard models might be distorted by these extreme values. So when choosing the distribution we have to pick the ones that can appropriately handle these large claims.

Now we check if there is a connection between the number of incidents and the geographical area:

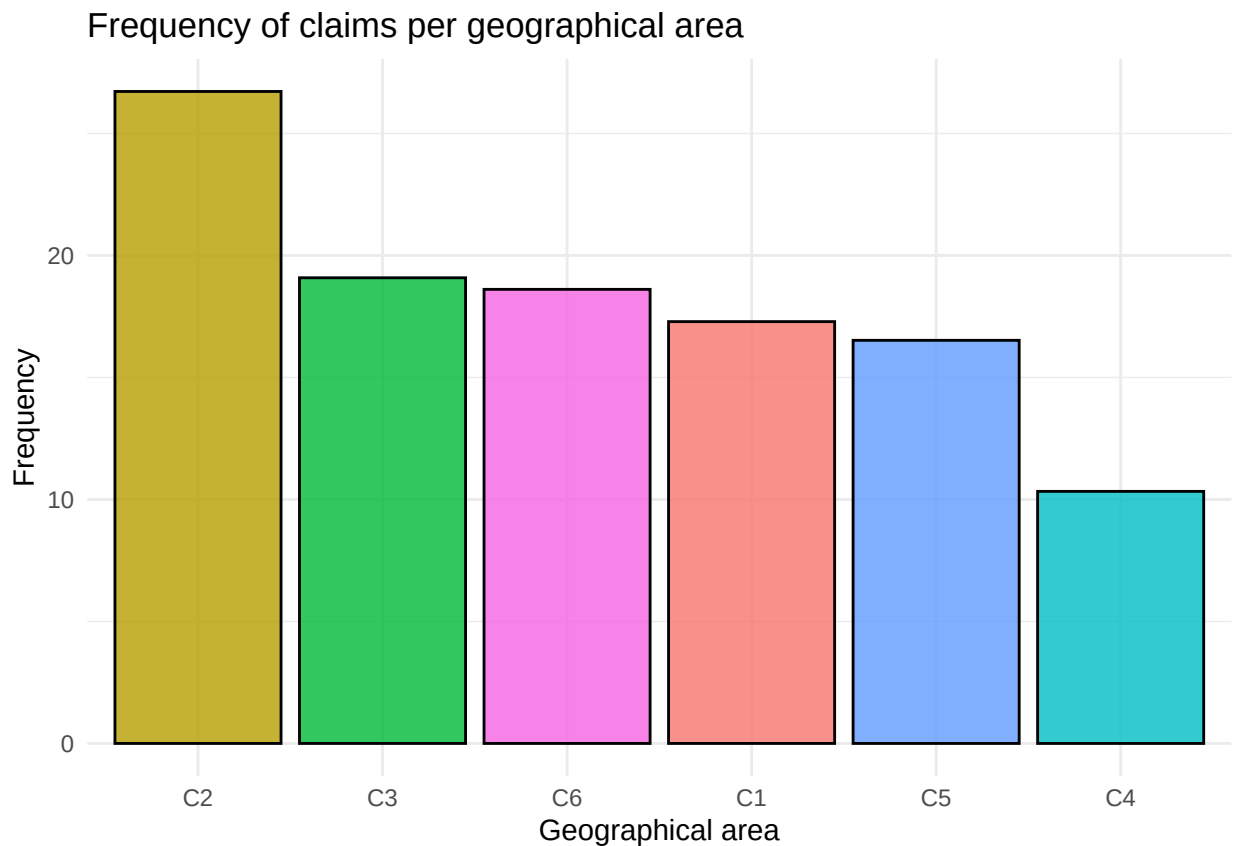
```
datageograp=dataclean %>%
  group_by(C_C) %>%
  summarise(
```

```

exposure=sum(exposure_time, na.rm=TRUE),
claims=sum(n_medical_services, na.rm=TRUE),
frequency=claims/exposure,
.groups='drop'
) %>%
drop_na(C_C)

ggplot(data=geograp, aes(x=reorder(C_C, -frequency), y=frequency, fill=C_C))+
  geom_col(color='black', alpha=0.8)+
  theme_minimal()+
  labs(
    title='Frequency of claims per geographical area',
    x='Geographical area',
    y='Frequency'
  )+
  theme(legend.position='none')

```



The bar chart, ordered in a descending way, shows **regional differences** in medical service utilization. We can see that the geographical area with the highest frequency is C2 followed by C3, C6, C1, C5 and C4, so C4 is the lowest one. Now we can hypothesize that claim frequency may be correlated with *localized environmental* and *climatic factors* such as temperature, UV index, altitude and atmospheric pressure. Based on the distribution in the reference map, we can assume the following geographical and climatic profiles for each cluster:

- C1 (Central Plateau/Meseta Central)**: located around Madrid, characterized by a continental Mediterranean temperature with cold winter and very hot summers.
- C2 (Canary Islands)**: a volcanic archipelago in the Atlantic Ocean with subtropical climate and

mountainous terrain.

-C3 (Mediterranean Coast and Southern Strip): characterized by a warm Mediterranean climate with hot summers and mild winters.

-C4 (Central and Northern Plateau): features a harsh continental climate with seasonal and daily temperature ranges, crossed by several mountain ranges.

-C5 (Eastern Inland and Southern Areas): a transitional zone blending both Mediterranean and mountainous climatic influences

-C6 (North and West Coast): characterized by an oceanic climate with abundant rainfall, high humidity, mountainous topography and mild temperature all year.

We now introduce the dataset with the *temperature characteristics*:

```
dataclimate=read_excel('Centroid values of the climate clusters.xlsx')
```

We now merge the two dataset but before we do a fast analysis of the new one:

```
summary(dataclimate)
```

```
##      C_C      Year      Temperature      Precipitation
## Length:18      Min.   :2017      Min.   :12.94      Min.   :0.2771
## Class :character 1st Qu.:2017      1st Qu.:13.94      1st Qu.:0.9332
## Mode  :character Median :2018      Median :16.07      Median :1.1540
##                      Mean  :2018      Mean  :16.23      Mean  :1.5082
##                      3rd Qu.:2019      3rd Qu.:17.74      3rd Qu.:1.7759
##                      Max.   :2019      Max.   :20.91      Max.   :3.6618
##      Wind      UV index      Atmospheric Pressure
## Min.   :2.637      Min.   :5.208      Min.   :923.6
## 1st Qu.:2.787      1st Qu.:7.518      1st Qu.:947.8
## Median :2.871      Median :7.987      Median :974.1
## Mean   :3.073      Mean   :7.681      Mean   :967.0
## 3rd Qu.:3.032      3rd Qu.:8.490      3rd Qu.:984.4
## Max.   :4.224      Max.   :8.834      Max.   :995.9
```

```
dataclean=dataclean %>%
  left_join(dataclimate, by=c('C_C'='C_C', 'period'='Year'))
dataclean=dataclean %>%
  rename(
    Pressure='Atmospheric Pressure',
    UV_index='UV index'
  )
```

To investigate the geographical differences, we add our core dataset with an external dataset containing climatic records for each geographic cluster. First, we merge the dataset (dataclimate) using `left_join`, keyed on both `C_C` and `period` match `Year` (`observation_year`). This ensure that each policy is matched with the *exact atmospheric conditions* of its respective region and year. The climatic features include:

-Temperature: average temperature in Celsius.

-Precipitation: average rainfall/snowfall.

-Wind: average wind speed.

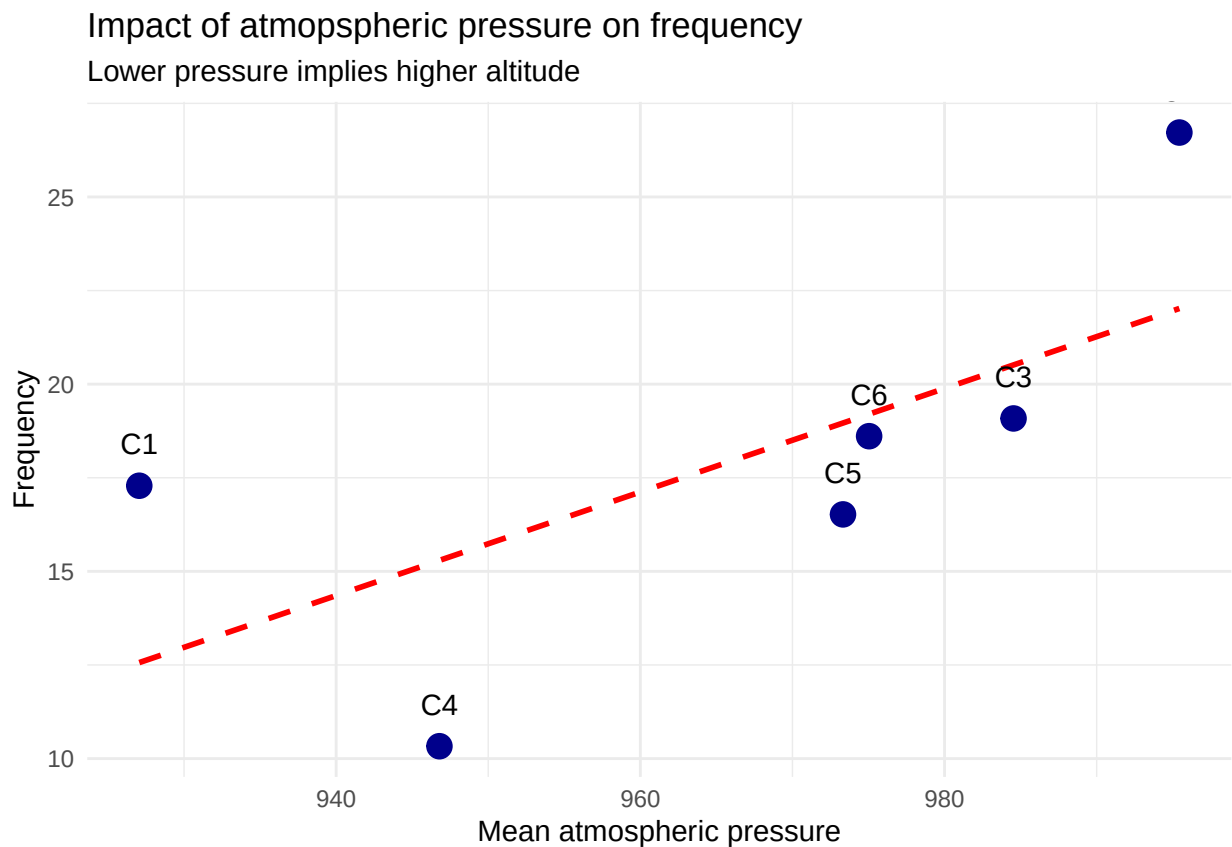
-UV Index: measure of ultraviolet radiation.

-Pressure: atmospheric pressure (inversely correlated with the altitude of the region).

```
dataclimateanalysis=dataclean %>%
  group_by(C_C) %>%
  summarise(
    exposure=sum(exposure_time, na.rm=TRUE),
    claims=sum(n_medical_services, na.rm=TRUE),
    frequency=claims/exposure,
    meanpressure=mean(Pressure, na.rm=TRUE),
    meantemp=mean(Temperature, na.rm=TRUE),
    meanuv=mean(UV_index, na.rm=TRUE),
    meanprecipitation=mean(Precipitation, na.rm=TRUE),
    meanwind=mean(Wind, na.rm=TRUE),
    .groups='drop'
  ) %>%
  drop_na(C_C)
```

Now we plot the ggplot:

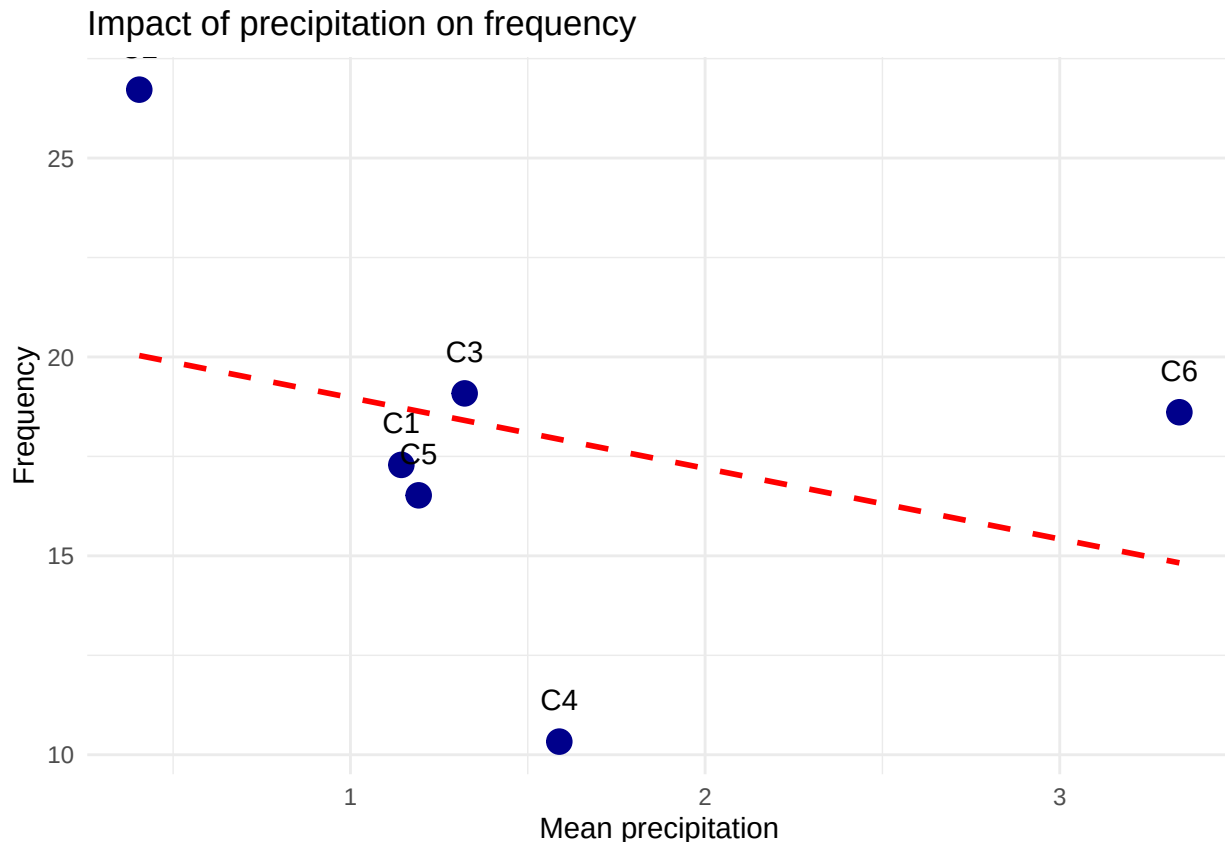
```
ggplot(dataclimateanalysis, aes(x=meanpressure, y=frequency))+ geom_point(color='darkblue', size=4)+
  geom_smooth(method='lm', se=FALSE, color='red', linetype='dashed')+
  geom_text(aes(label=C_C),vjust=-1.5)+
  theme_minimal()+labs(
    title='Impact of atmopspheric pressure on frequency',subtitle='Lower pressure implies higher altitude',
    x='Mean atmospheric pressure',
    y='Frequency'
  )
```



The plot illustrates a positive correlation between mean atmospheric pressure and claim frequency. Atmospheric pressure is inversely related to altitude: regions with *lower pressure correspond to higher elevations* (such as Pyrenees in C4) and viceversa. Noting that C1 is an exception, indeed although the low pressure, it is a very urbanized area also because of the presence of Madrid. This suggests that mountainous regions which are usually less densely populated and have harder to reach medical facilities show a lower frequency in claim. On the other hand, the more urbanized and low-altitude ones have a much higher volume of medical services utilized.

Now we conduct a similar analysis but not only looking at the relationship of pressure and frequency but also other variables. We can start with precipitation:

```
ggplot(dataclimateanalysis, aes(x=meanprecipitation, y=frequency))+ geom_point(color='darkblue', size=4)+
  geom_smooth(method='lm', se=FALSE, color='red', linetype='dashed')+
  geom_text(aes(label=C_C), vjust=-1.5)+
  theme_minimal()+labs(
    title='Impact of precipitation on frequency',
    x='Mean precipitation',
    y='Frequency'
  )
```



The plot shows a weak negative correlation between mean precipitation and claim frequency. We might think that regions with less rainfall experience a higher frequency of medical claims. However, this relationship is confounded by demographic and geographical realities, as underlined by two majors outliers:

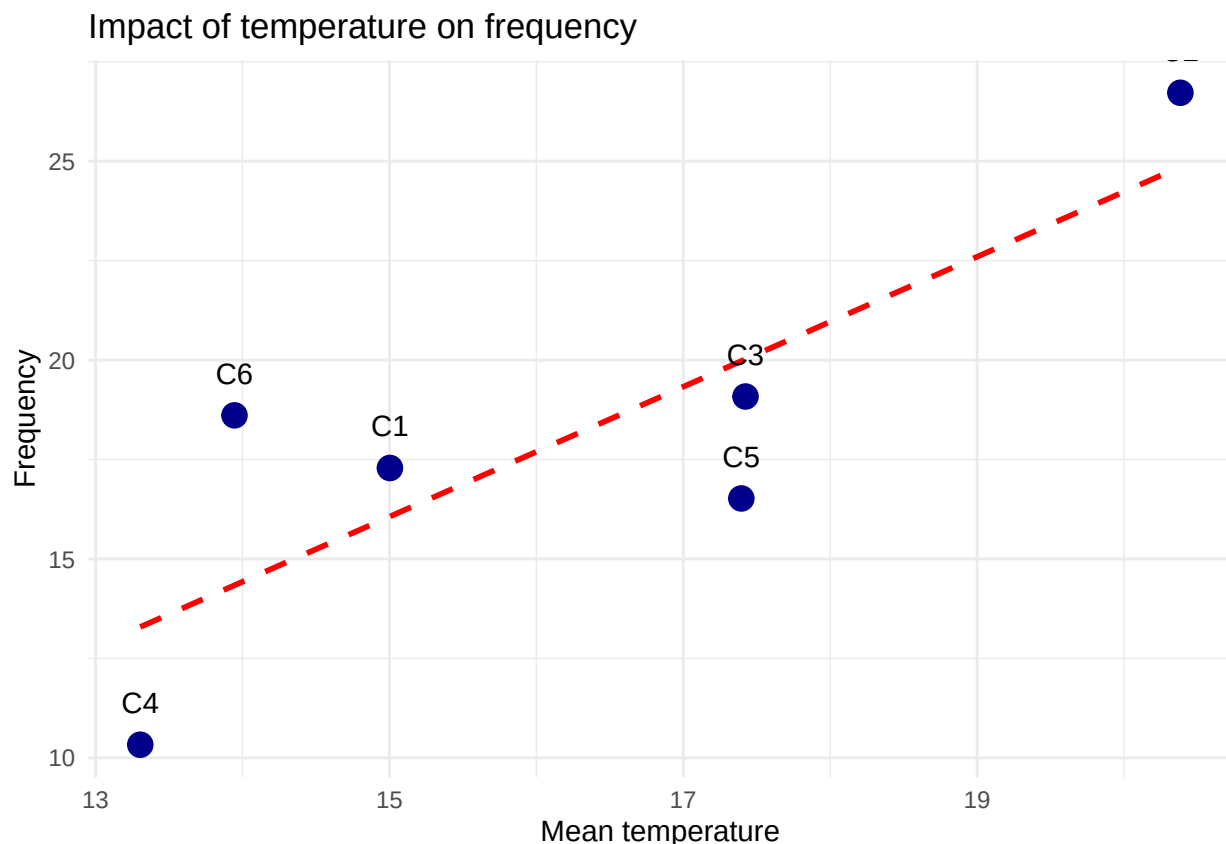
- Cluster C6 (North and West Coast):** despite having the highest rain precipitation, C6 maintains a high frequency. This happens because coastal areas are densely populated and urbanized.
- Cluster C4 (Central and Northern Plateau):** C4 has moderate precipitations but also the lowest claim frequency. This is connected to our previous assumption, the mountainous and sparsely populated C4 area

is the primary driver of low medical service utilization.

In conclusion, while a statistical trend exist, *precipitation* is behaving as a **proxy for region type** rather than a driver of medical claims.

Now we can study the connection with temperature:

```
ggplot(dataclimateanalysis, aes(x=meantemp, y=frequency))+ geom_point(color='darkblue', size=4)+
  geom_smooth(method='lm', se=FALSE, color='red', linetype='dashed')+
  geom_text(aes(label=C_C),vjust=-1.5)+
  theme_minimal()+labs(
    title='Impact of temperature on frequency',
    x='Mean temperature',
    y='Frequency'
  )
```



The plot shows a *positive linear correlation* between **mean temperature** and **frequency of the medical claims**. The warmest regions (C2 and C3) correspond to highly urbanized, densely populated coastal areas with accessible health care infrastructures. In opposite, the coldest region (C4) represent the one with harsh and mountainous inland plateau. Temperature serves as a geographic proxy: coastal climates positively correlate with higher population densities and easier access to medical services, which drives up the frequency of claims.

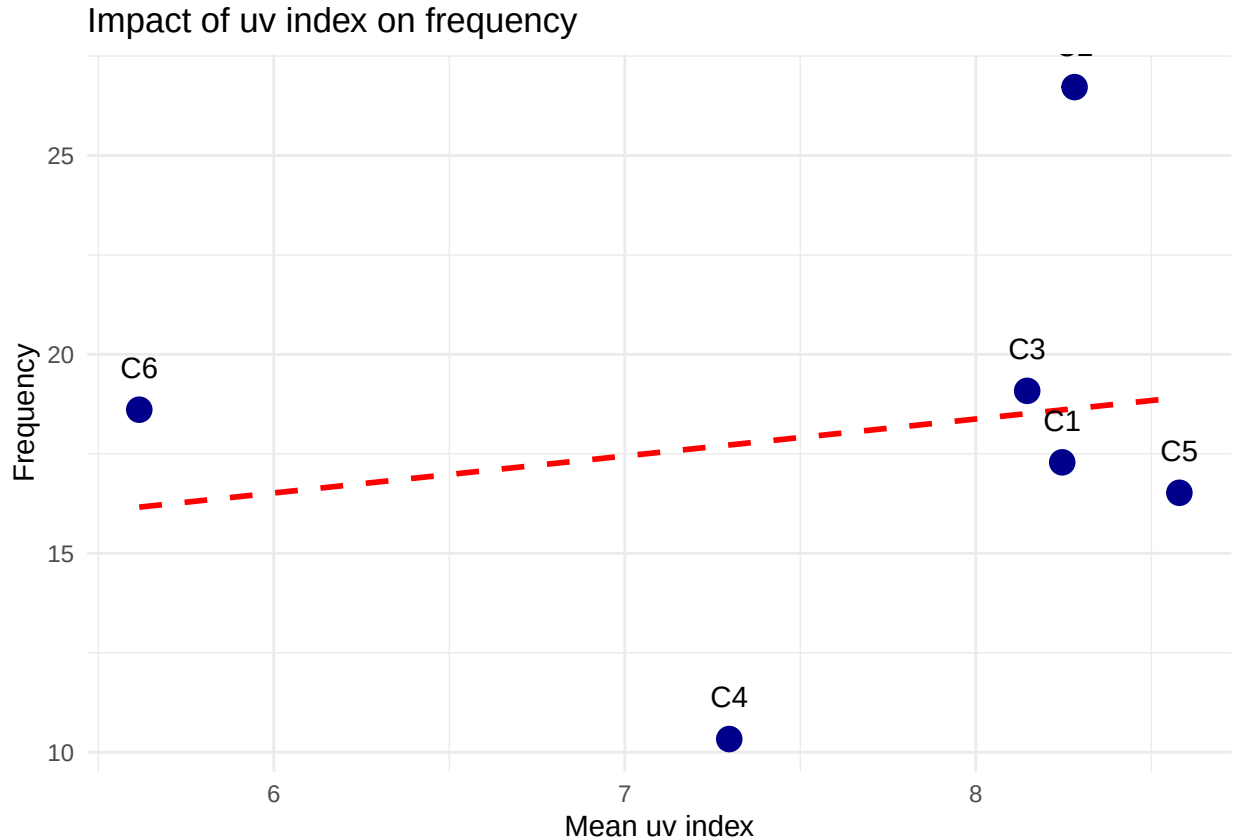
Now we study if UV index is affecting our dataset or the number of claims:

```
ggplot(dataclimateanalysis, aes(x=meanuv, y=frequency))+ geom_point(color='darkblue', size=4)+
  geom_smooth(method='lm', se=FALSE, color='red', linetype='dashed')+
  geom_text(aes(label=C_C),vjust=-1.5)+
  theme_minimal()+labs(
```

```

title='Impact of uv index on frequency',
x='Mean uv index',
y='Frequency'
)

```



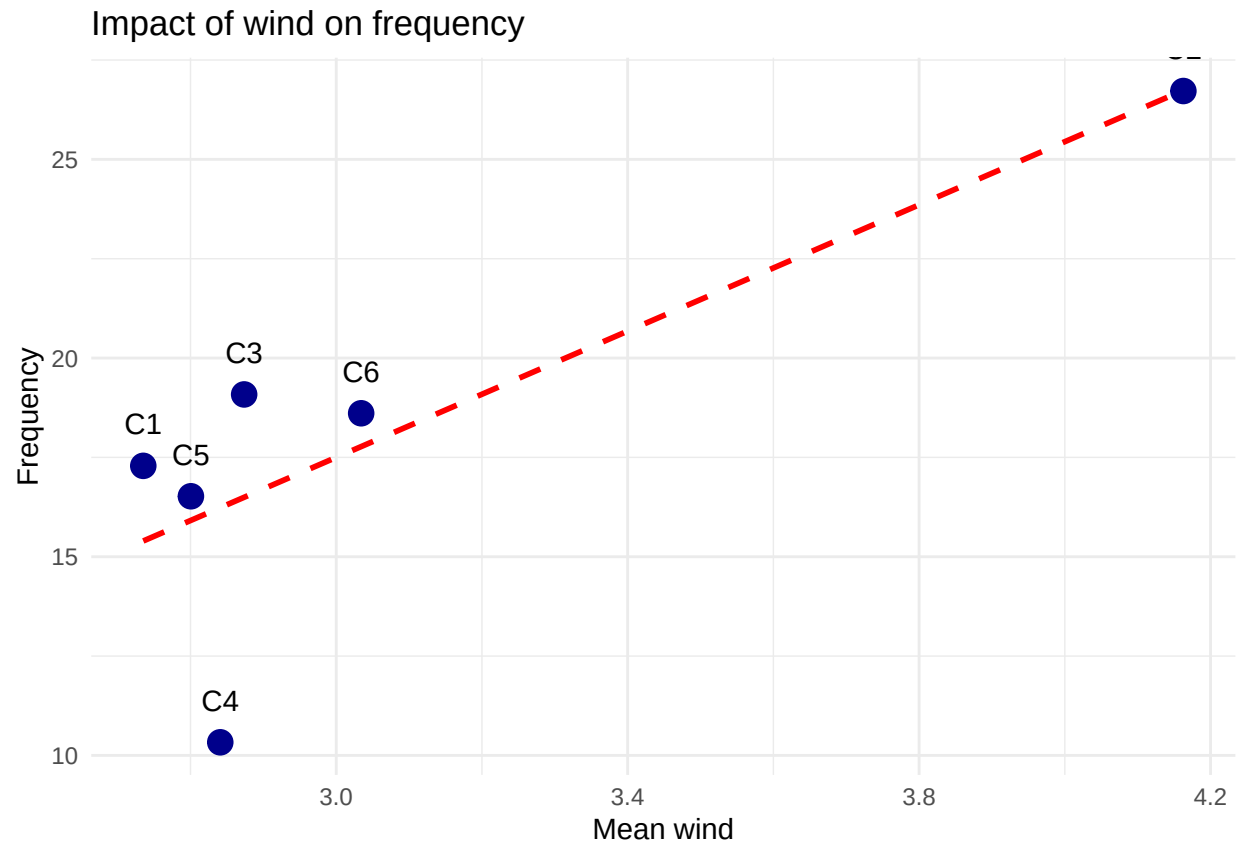
In this case we can see a nearly flat regression line, so there is *no meaningful correlation* between the mean UV index and the frequency of medical claims. Indeed the data points are scattered regardless of the UV index. The lack of correlation confirms that specific meteorological variables like the UV index do not directly influence a population's overall medical service utilization.

We do the same with wind:

```

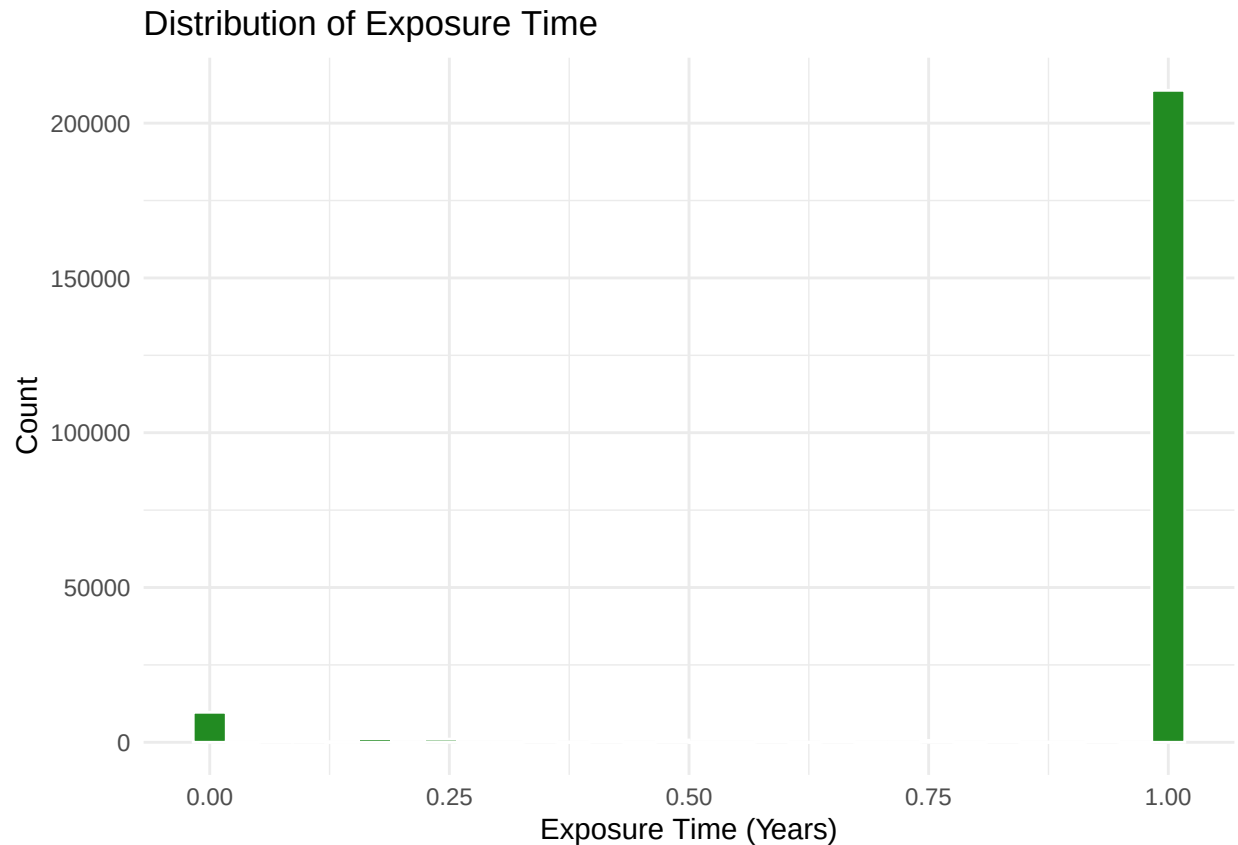
ggplot(dataclimateanalysis, aes(x=meanwind, y=frequency))+ geom_point(color='darkblue', size=4)+
geom_smooth(method='lm', se=FALSE, color='red', linetype='dashed')+
geom_text(aes(label=C_C),vjust=-1.5)+
theme_minimal()+labs(
  title='Impact of wind on frequency',
  x='Mean wind',
  y='Frequency'
)

```



At first glance, the plot might suggest a positive correlation between mean wind speed and claim frequency, but in reality under an actuarial and medical viewpoint this seems highly improbable. This is mainly driven by a **single leverage point** in the top right part of the plot (C2). Indeed Canary Islands are characterized by constant strong winds, while also having one of the highest claims frequency because highly populated. Then if the single observation was removed, the remaining clusters would display no meaningful correlation between wind speed and medical claims. In conclusion, we can classify it as a spurious correlation. Wind is not a risk driver but a coincidental climatic feature of the highest frequency region.

```
ggplot(dataclean, aes(x=exposure_time)) +
  geom_histogram(bins=30, fill='forestgreen', color='white') +
  theme_minimal() +
  labs(title = 'Distribution of Exposure Time', x='Exposure Time (Years)', y='Count')
```



The histogram shows the distribution of exposure time across the portfolio. The distribution has a peak at 1, this means that the majority of the insured held an active policy for the entire duration of the observed year. The small bars between 0 and 1 represent fraction exposures so new policies or cancellations during the year.

We visually summarize the geographic distribution of medical service utilization with a map of Spain using the sf and mapSpain packages.

```
library(sf)
library(mapSpain)
library(dplyr)
library(ggplot2)
```

```
datageograp=ataclean %>%
  group_by(C_C) %>%
  summarise(
    exposure=sum(exposure_time, na.rm = TRUE),
    claims=sum(n_medical_services, na.rm = TRUE),
    frequency = claims / exposure,
    .groups='drop'
  ) %>%
  drop_na(C_C)

spain_ccaa=esp_get_ccaa()

cluster_mapping=data.frame(
```

```

ine.ccaa.name = c("Andalucía", "Aragón", "Principado de Asturias", "Illes Balears",
                  "Canarias", "Cantabria", "Castilla y León", "Castilla-La Mancha",
                  "Cataluña", "Comunitat Valenciana", "Extremadura", "Galicia",
                  "Comunidad de Madrid", "Región de Murcia", "Comunidad Foral de Navarra",
                  "País Vasco", "La Rioja", "Ceuta", "Melilla"),
C_C = c("C3", "C4", "C6", "C3",
        "C2", "C6", "C4", "C1",
        "C5", "C3", "C1", "C6",
        "C1", "C3", "C4",
        "C6", "C4", "C3", "C3")
)

spain_map_freq=spain_ccaa %>%
  left_join(cluster_mapping, by="ine.ccaa.name") %>%
  left_join(datageograp, by="C_C")

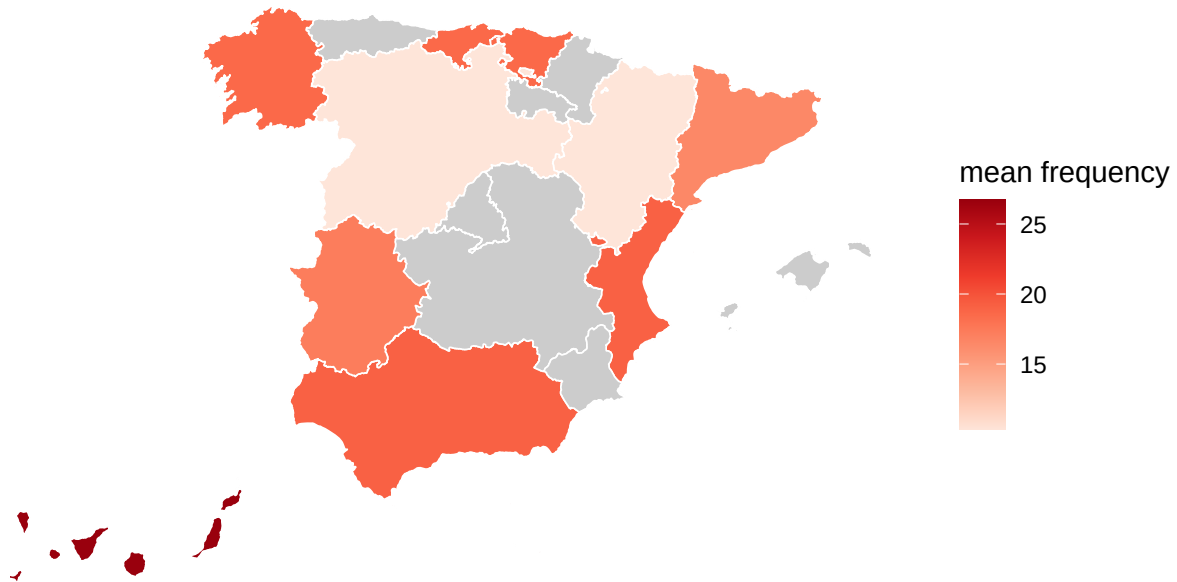
ggplot(spain_map_freq) +
  geom_sf(aes(fill=frequency), color = "white", size = 0.3) +

  scale_fill_distiller(palette = "Reds", direction = 1, na.value="gray80") +
  theme_minimal() +
  labs(title="Maps of frequency for area",
       subtitle="Darkest colors implies a bigger usage of medical services",
       fill="mean frequency") +
  theme(panel.grid=element_blank(),
        axis.text=element_blank(),
        axis.ticks=element_blank())

```


Maps of frequency for area

Darkest colors implies a bigger usage of medical services



We face some *limitations*:

-**Heuristic Mapping**: the original dataset provides anonymized climatic clusters rather than the standard administrative regions. While this is a geographical approximation, it successfully captures the macro-regional trends in medical services utilization.

-**Missing Data**: any regions in gray on the map represent areas where `C_C` was missing (NA) for the insured clients.

Modelling the claim frequency

Now we focus on modelling the **frequency** of claims.

As highlighted in the exploratory data analysis, the target variable is `n_medical_services` (number of claims per policy), and the exposure is `exposure_time` (measured in years).

Since the variance of `n_medical_services` is much larger than its mean (overdispersion), a standard Poisson model would be inappropriate as anticipated before. For this reason we decided to adopt two different modelling techniques:

1. **Negative Binomial GLM** – extends the Poisson model by adding a dispersion parameter to handle overdispersion.
2. **Gradient Boosting Machine (GBM)** with Poisson loss and offset – a non-parametric tree-based method that can capture non-linear effects and interactions.

We will train the models on 80% of the data and evaluate them on the remaining 20% using appropriate metrics (MAE, RMSE, calibration plots).

First of all we need to divide our dataset in training data and testing data:

```
df_mod=dataclean %>%
  filter(!is.na(exposure_time), exposure_time > 0,
         !is.na(age), !is.na(gender), !is.na(type_product), !is.na(C_C)) %>%
  mutate(Claims = n_medical_services,
         Exposure = exposure_time)

set.seed(123)
n=nrow(df_mod)
train_idx=sample(seq_len(n), size = floor(0.8 * n))
train=df_mod[ train_idx, ]
test=df_mod[-train_idx, ]
```

Proof of non-suitability of the Poisson model

Before going to the two models proposed above we need to give a proof that the Poisson one is not suitable for our case. Our goal is to estimate the rate λ that is defined as:

$$\lambda = \frac{\mathbb{E}[\text{Claims}]}{\text{Exposure}}$$

In reality we want to use the logarithm of it (to not have negative values) so it becomes:

$$\log(\lambda) = \beta_0 + \beta_1 x_1 + \dots$$

From this we obtain:

$$\log(\mathbb{E}[\text{Claims}]) = \log(\text{Exposure}) + \beta_0 + \beta_1 x_1 + \dots$$

We want to compare two different approaches for the Poisson model: Geographic Approach (we use C_C variable) and Climatic Approach (using physical variable). We do this comparison to stay in line with the analysis done in the previous sector, as we saw that pressure and temperature are proportional to frequency while the precipitation is inversely proportional. Then we compare it in order to choose the model that is more suitable for our case.

```
# Model 1: Geographic Approach (using C_C cluster as a proxy)
# This model assumes that the risk is tied to the specific region.
glm_pois_geo=glm(Claims ~ age + gender + type_product + C_C +
                 offset(log(Exposure)),
                 family = poisson(link = "log"),
                 data = train)

# Model 2: Climatic Approach (using physical sensors)
# This model tests if the risk is driven by specific weather conditions
# (Temperature, Pressure, Wind) instead of just "location".
glm_poi_clim=glm(Claims ~ age + gender + type_product + Temperature + Pressure + Wind +
                 offset(log(Exposure)),
                 family = poisson(link = "log"),
                 data = train)

# Compare performance using AIC and Deviance
# Lower AIC indicates a better balance between fit and complexity.
```

```

model_comp=data.frame(
  Model = c("Geographic (C_C)", "Climatic (Temp/Pres/Wind)"),
  AIC = c(AIC(glm_pois_geo), AIC(glm_poi_clim)),
  Deviance = c(glm_pois_geo$deviance, glm_poi_clim$deviance)
)

print(model_comp)

```

```

##              Model      AIC Deviance
## 1      Geographic (C_C) 4603346 4063217
## 2 Climatic (Temp/Pres/Wind) 4630325 4090201

```

The geographic cluster C_C captures the variance in the data better than the three specific climate sensors combined. So we continue the proof of non-suitability with that specific model.

```
summary(glm_pois_geo)
```

```

##
## Call:
## glm(formula = Claims ~ age + gender + type_product + C_C + offset(log(Exposure)),
##      family = poisson(link = "log"), data = train)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -5.478e+00 1.059e-01 -51.71 <2e-16 ***
## age          1.910e-02 2.785e-05  686.00 <2e-16 ***
## genderM      -2.082e-01 1.203e-03 -173.04 <2e-16 ***
## type_productI 7.454e+00 1.060e-01  70.31 <2e-16 ***
## type_productP 7.526e+00 1.059e-01  71.06 <2e-16 ***
## type_productS 7.441e+00 1.059e-01  70.27 <2e-16 ***
## C_CC2         3.505e-01 3.425e-03 102.34 <2e-16 ***
## C_CC3         1.393e-01 1.963e-03  70.97 <2e-16 ***
## C_CC4        -4.033e-01 2.955e-03 -136.47 <2e-16 ***
## C_CC5        -6.102e-02 3.972e-03  -15.36 <2e-16 ***
## C_CC6         1.597e-01 2.146e-03  74.41 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##      Null deviance: 4977530  on 167827  degrees of freedom
## Residual deviance: 4063217  on 167817  degrees of freedom
## AIC: 4603346
##
## Number of Fisher Scoring iterations: 6

```

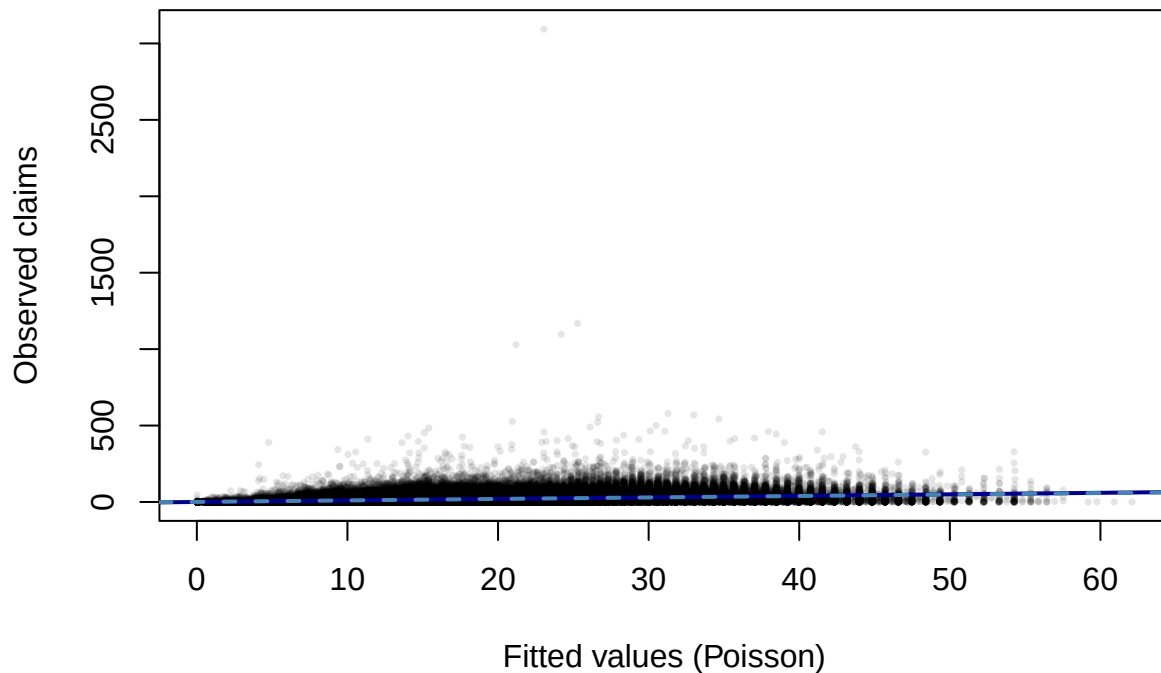
```

plot(glm_pois_geo$fitted.values, train$Claims,
     xlab = "Fitted values (Poisson)",
     ylab = "Observed claims",
     main = "Poisson GLM: fitted vs observed",
     pch = 16, col = rgb(0, 0, 0, 0.1), cex = 0.5)
abline(0, 1, col = "darkblue", lwd = 2)

```

```
abline(lm(train$Claims ~ glm_pois_geo$fitted.values),
       col = "steelblue", lwd = 2, lty = 2)
```

Poisson GLM: fitted vs observed



```
# Overdispersion: ratio deviance / df
disp_ratio <- deviance(glm_pois_geo) / df.residual(glm_pois_geo)
cat("Residual deviance:", round(deviance(glm_pois_geo), 0), "\n")
```

```
## Residual deviance: 4063217
```

```
cat("Degrees of freedom:", df.residual(glm_pois_geo), "\n")
```

```
## Degrees of freedom: 167817
```

```
cat("Dispersion ratio:", round(disp_ratio, 2), "\n")
```

```
## Dispersion ratio: 24.21
```

```
# Formal test for overdispersion (AER)
library(AER)
dispersiontest(glm_pois_geo)
```

```
##
```

```
## Overdispersion test
##
## data: glm_pois_geo
## z = 12.128, p-value < 2.2e-16
## alternative hypothesis: true dispersion is greater than 1
## sample estimates:
## dispersion
## 38.27955
```

In this model we are predicting the frequency using the geographical variable `C_C`. From the output we can see that:

1. **Residual deviance** = 4,063,217
2. **Degrees of freedom** = 167,817
3. **Ratio deviance / df** = 24.2 ($\gg 1$)

This indicates **strong overdispersion** (as already observed in the previous section of descriptive analysis). Therefore, the Poisson model is not appropriate.

We can now move on to a model that **accounts for overdispersion**, such as the **Negative Binomial**.

Negative Binomial GLM

As we saw in the Poisson specification, we use the same relationship here:

$$\log(E[\text{Claims}]) = \log(\text{Exposure}) + \beta_0 + \beta_1 x_1 + \dots$$

So, also in the Negative Binomial specification, we want to use the offset term for the variable $\log(\text{Exposure})$. The main difference between the two models is that in the Poisson model, the variance and the mean are equal, while in the Negative Binomial model, the variance is defined as:

$$\text{Var}[\text{Claims}] = E[\text{Claims}] + \frac{E[\text{Claims}]^2}{\theta}$$

This allows us to look at the overdispersion (which was not possible in the Poisson model).

```
# Negative Binomial model (accounts for overdispersion)
library(MASS)

glm_nb_geo=glm.nb(Claims ~ age + gender + type_product + C_C +
                  offset(log(Exposure)),
                  data = train)

glm_nb_clim=glm.nb(Claims ~ age + gender + type_product +
                  Temperature + Pressure + Wind +
                  offset(log(Exposure)),
                  data = train)

model_comp_nb=data.frame(
  Model    = c("Geographic (C_C)", "Climatic (Temp/Pres/Wind)"),
  AIC      = c(AIC(glm_nb_geo),      AIC(glm_nb_clim)),
  Deviance = c(deviance(glm_nb_geo), deviance(glm_nb_clim))
)
print(model_comp_nb)
```

```
##           Model      AIC Deviance
## 1      Geographic (C_C) 1169963 186439.9
## 2 Climatic (Temp/Pres/Wind) 1170950 186472.4
```

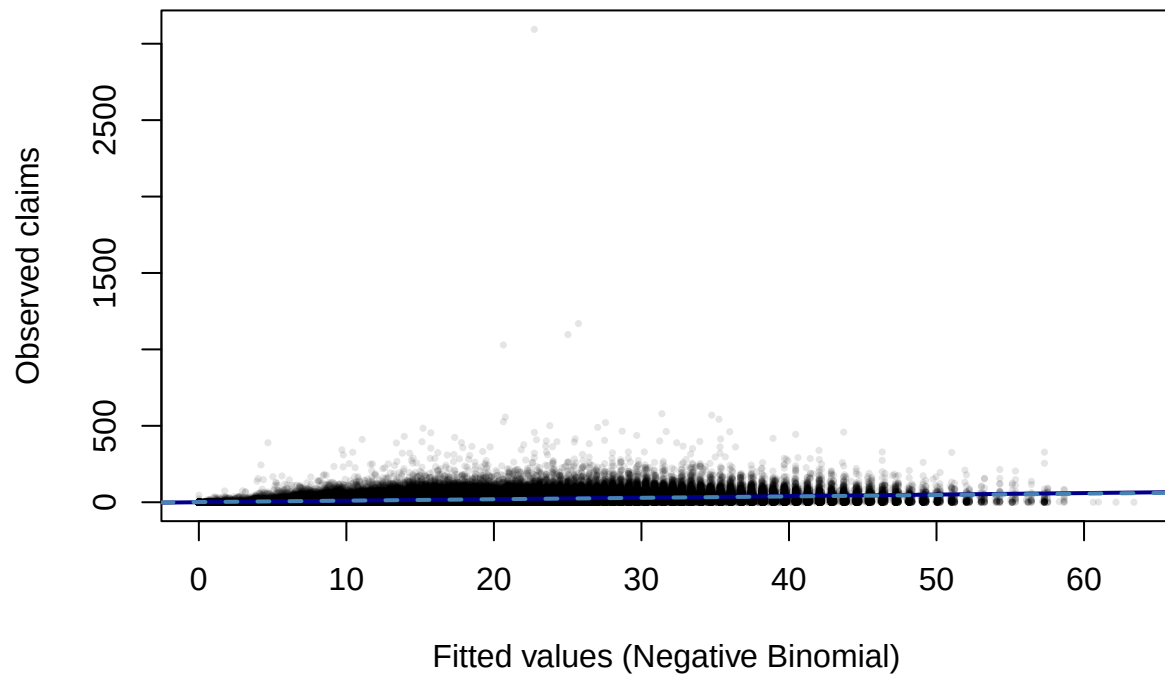
Also in this case the C_C variable is preferred.

```
summary(glm_nb_geo)
```

```
##
## Call:
## glm.nb(formula = Claims ~ age + gender + type_product + C_C +
##       offset(log(Exposure)), data = train, init.theta = 0.4833693438,
##       link = log)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -5.4676196  0.1080398 -50.607   <2e-16 ***
## age          0.0193570  0.0001617 119.744   <2e-16 ***
## genderM     -0.2698370  0.0073642 -36.642   <2e-16 ***
## type_productI 7.2225392  0.1116544  64.687   <2e-16 ***
## type_productP 7.5009673  0.1074790  69.790   <2e-16 ***
## type_productS 7.4484260  0.1071207  69.533   <2e-16 ***
## C_CC2        0.3294792  0.0241291  13.655   <2e-16 ***
## C_CC3        0.1514053  0.0119489  12.671   <2e-16 ***
## C_CC4       -0.3972675  0.0154486 -25.716   <2e-16 ***
## C_CC5       -0.0496823  0.0233672  -2.126    0.0335 *
## C_CC6        0.1743355  0.0129944  13.416   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for Negative Binomial(0.4834) family taken to be 1)
##
##      Null deviance: 229675  on 167827  degrees of freedom
## Residual deviance: 186440  on 167817  degrees of freedom
## AIC: 1169963
##
## Number of Fisher Scoring iterations: 1
##
##              Theta:  0.48337
##              Std. Err.:  0.00178
##
## 2 x log-likelihood:  -1169938.82700
```

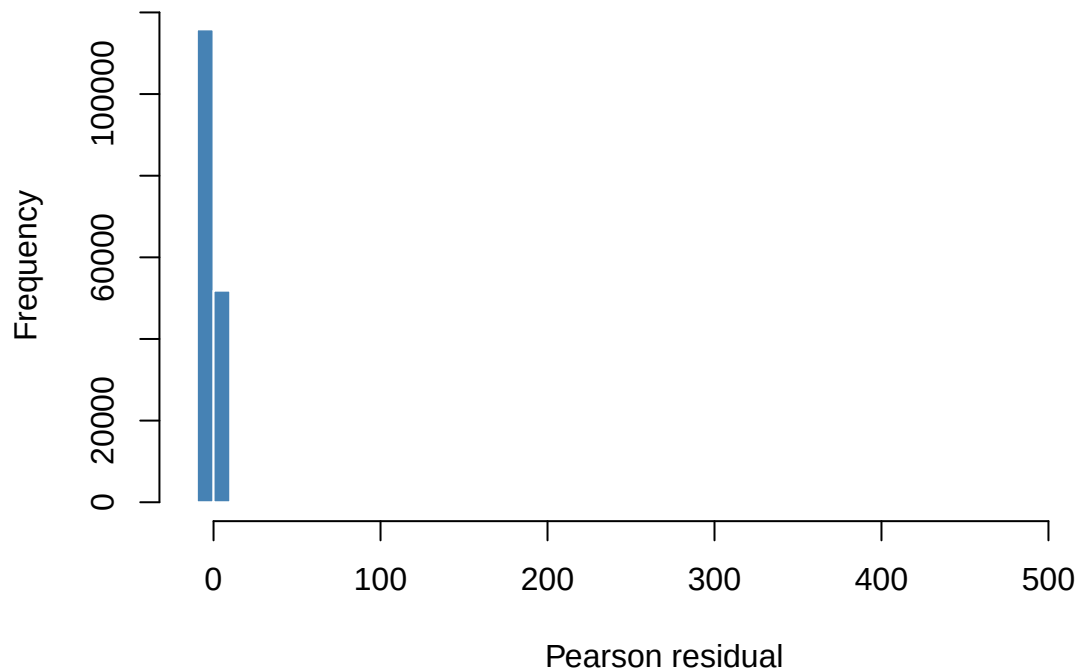
```
# Fitted vs Observed
plot(glm_nb_geo$fitted.values, train$Claims,
     xlab = "Fitted values (Negative Binomial)",
     ylab = "Observed claims",
     main = "Negative Binomial GLM: fitted vs observed",
     pch = 16, col = rgb(0, 0, 0, 0.1), cex = 0.5)
abline(0, 1, col = "darkblue", lwd = 2)
abline(lm(train$Claims ~ glm_nb_geo$fitted.values),
     col = "steelblue", lwd = 2, lty = 2)
```

Negative Binomial GLM: fitted vs observed



```
pearson_res=residuals(glm_nb_geo, type = "pearson")
hist(pearson_res, breaks = 60, col = "steelblue", border = "white",
     main = "Pearson residuals - Negative Binomial",
     xlab = "Pearson residual")
```

Pearson residuals — Negative Binomial



```
disp_nb=deviance(glm_nb_geo) / df.residual(glm_nb_geo)
cat("Theta (dispersion parameter):", round(glm_nb_geo$theta, 4), "\n")
```

```
## Theta (dispersion parameter): 0.4834
```

```
cat("Residual deviance:", round(deviance(glm_nb_geo), 0), "\n")
```

```
## Residual deviance: 186440
```

```
cat("Degrees of freedom:", df.residual(glm_nb_geo), "\n")
```

```
## Degrees of freedom: 167817
```

```
cat("Dispersion ratio:", round(disp_nb, 3), "\n")
```

```
## Dispersion ratio: 1.111
```

COEFFICIENTS OF THE GEOGRAPHIC NEGATIVE BINOMIAL MODEL

All predictors are statistically significant (p-values < 2e-16 apart from C_CC5 that is slightly significant).

The coefficients are:

- for age is positive , older policyholders tend to have higher claim frequencies.
- males have a negative coefficient (-0.2082)

- for the climatic areas, we can see that C2 and C3 have positive coefficients while C4 has a strong negative coefficient (-0.4033). This is consistent with the exploratory analysis.

The output shows:

- **Theta** = 0.483 (significantly different from infinity, which would correspond to Poisson).
- **Residual deviance** = 186,440 on 167,817 df → ratio=1.11, close to 1, indicating a good fit.
- **AIC** is not directly comparable with Poisson because of different likelihoods, but the drop in deviance confirms that the Negative Binomial is far more appropriate.

Model evaluation on the test set

We compute predictions on the test set and evaluate the Negative Binomial model using MAE, RMSE, and a calibration plot.

Managerial point of view:

From a managerial point of view using a Negative Binomial instead of a Poisson means that we are looking at the true variability in claim frequency, and it reduces the risk of underpricing (as the variance is » than the mean).

IMPORTANT= We can see that the coefficients for the type of products I,P,S is very high in comparison with the reference category D. It means that that types of product have higher frequency expectancy.

```
pred_nb=predict(glm_nb_geo, newdata = test, type = "response")
```

```
mae_nb=mean(abs(test$Claims - pred_nb))
rmse_nb=sqrt(mean((test$Claims - pred_nb)^2))
```

```
dev_nb=2 * sum(
  ifelse(test$Claims > 0,
    test$Claims * log(test$Claims / pred_nb),
    0) - (test$Claims - pred_nb)
)
```

```
cat("Negative Binomial - test set:\n")
```

```
## Negative Binomial - test set:
```

```
cat("  MAE:      ", round(mae_nb, 4), "\n")
```

```
##  MAE:      16.1646
```

```
cat("  RMSE:      ", round(rmse_nb, 4), "\n")
```

```
##  RMSE:      26.3775
```

```
cat("  Deviance: ", round(dev_nb, 0), "\n")
```

```
##  Deviance: 1024154
```

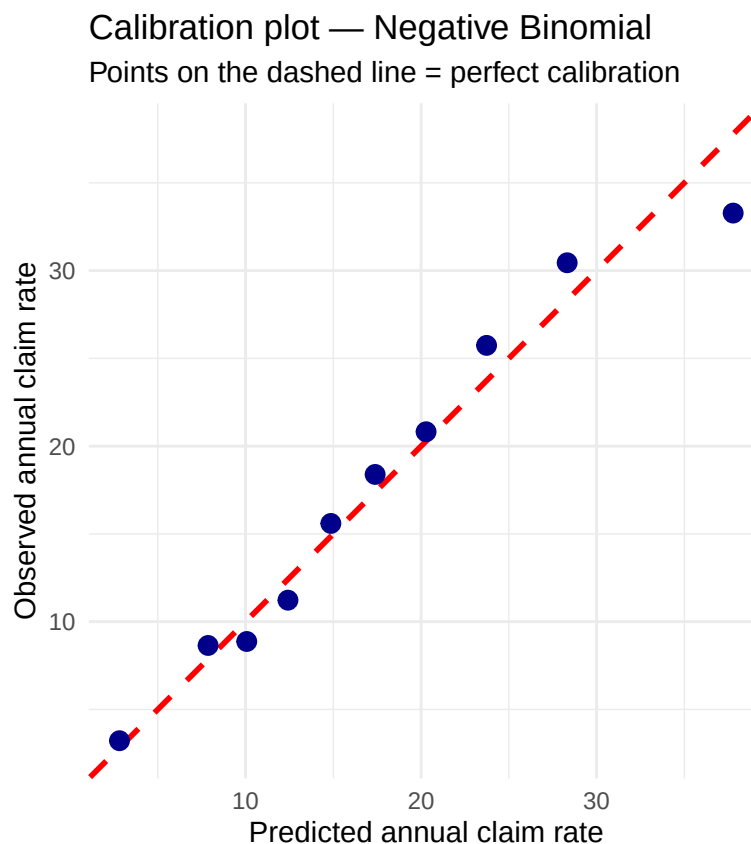
```

calib_nb=test %>%
  mutate(pred = pred_nb,
         decile = ntile(pred, 10)) %>%
  group_by(decile) %>%
  summarise(
    observed = sum(Claims),
    expected = sum(pred),
    exposure = sum(Exposure),
    .groups = "drop"
  ) %>%
  mutate(obs_rate = observed / exposure,
         exp_rate = expected / exposure)

lim_nb=range(c(calib_nb$obs_rate, calib_nb$exp_rate))

ggplot(calib_nb, aes(x = exp_rate, y = obs_rate)) +
  geom_abline(linetype = "dashed", color = "red", linewidth = 1) +
  geom_point(size = 3, color = "darkblue") +
  coord_fixed(xlim = lim_nb, ylim = lim_nb) +
  theme_minimal() +
  labs(title = "Calibration plot - Negative Binomial",
       subtitle = "Points on the dashed line = perfect calibration",
       x = "Predicted annual claim rate",
       y = "Observed annual claim rate")

```



From the output, the MAE is 16.16. This means the model predicts the average number of claims quite well,

but with high variability. The RMSE is higher (26.38), so predictions for policies with many claims are less accurate. The deviance is 1,024,154. The calibration plot shows that most points are close to the diagonal line, which means the model is well calibrated.

Gradient Boosting Machine (GBM)

The big difference in the GBM in comparison with the GLM is that this is a non-parametric method and so it is not based only on the linear regression with β .

The function that we obtain, it will be more complex and we define it as $F(x)$. So our final formula in this case will be:

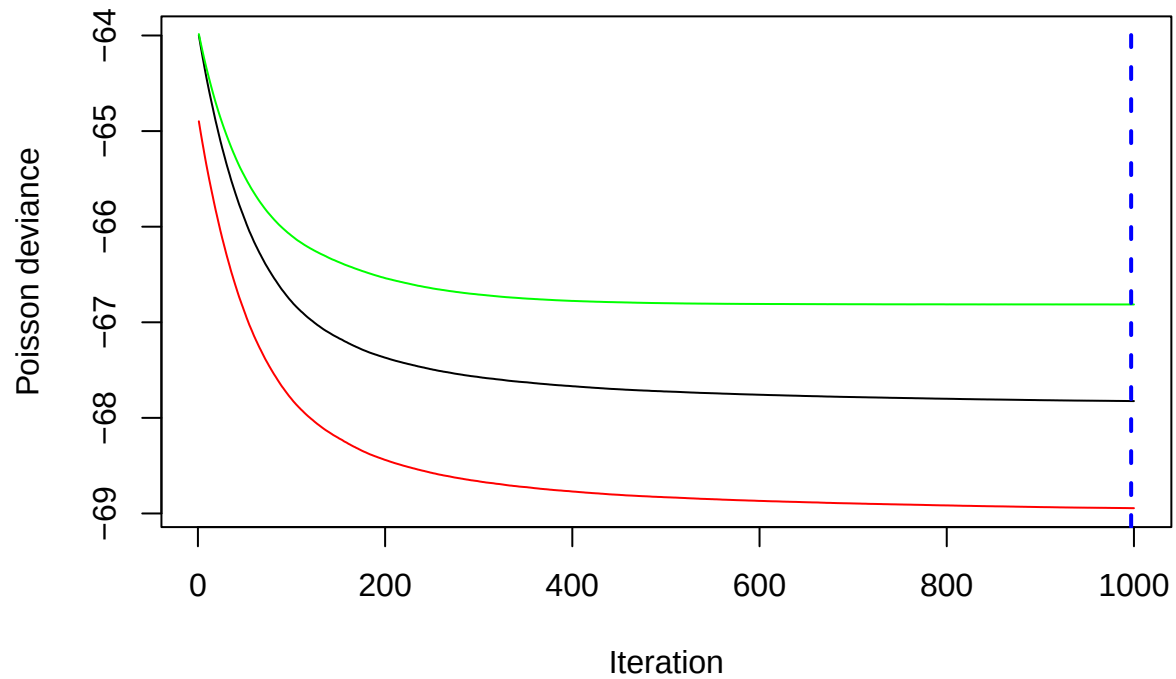
$$\log(E[Claims]) = \log(Exposure) + F(x_1, x_2, \dots)$$

We can notice that we have always the offset term (but we describe better our algorithm below).

```
library(gbm)
train$logExposure=log(train$Exposure)
test$logExposure=log(test$Exposure)
train$C_C=as.factor(train$C_C)
test$C_C=as.factor(test$C_C)
set.seed(123)
gbm_train=gbm(Claims ~ age + gender + type_product + C_C +
               offset(logExposure),
               data = train,
               distribution = "poisson",
               n.trees = 1000,
               shrinkage = 0.01,
               interaction.depth = 3,
               bag.fraction = 0.5,
               train.fraction = 0.5,
               n.minobsinnode = 10000,
               cv.folds = 3,
               keep.data = TRUE,
               verbose = FALSE,
               n.cores = 1)
```

```
## CV: 1
## CV: 2
## CV: 3
```

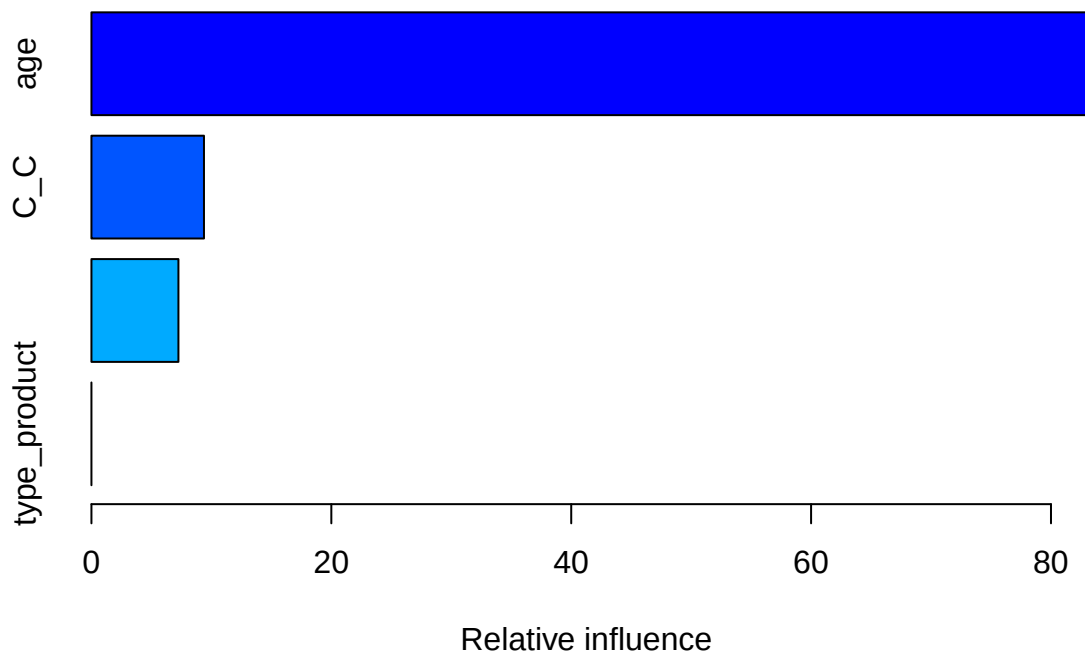
```
best.iter=gbm.perf(gbm_train, method = "cv")
```



```
print(paste("Best number of trees:", best.iter))
```

```
## [1] "Best number of trees: 997"
```

```
summary(gbm_train, n.trees = best.iter)
```



```
##               var   rel.inf
## age           age 83.365544
## C_C           C_C  9.382321
## gender        gender 7.252136
## type_product type_product 0.000000
```

We fit a Gradient Boosting Machine with Poisson loss, using the same covariates as the Negative Binomial model and an offset for exposure.

Key parameters: `n.trees = 1000`, `shrinkage = 0.01`, `interaction.depth=3`, `train.fraction = 0.5`, `n.minobsinnode = 10000`. The optimal number of trees identified via 3-fold cross-validation was 997. This confirms that our initial parameter of `n.trees = 1000` is highly appropriate, capturing nearly all the predictive power without the need for additional, computationally expensive iterations.

We can see from the output that the variable **age** has the highest weight in predicting the frequency (83.37%), followed by the **C_C** with 9.38% and **gender** with 7.25%. This can be shown in the later graph. Here we can see that the effect of the **age** is not constant, for example for female the young female policyholders have a higher frequency (can be correlated with maternity).

```
# here we add manually test$Exposure because the first output was giving a warning that was adding the
pred_gbm=predict(gbm_train, newdata = test, n.trees = best.iter, type = "response") * test$Exposure

# Metrics
mae_gbm=mean(abs(test$Claims - pred_gbm))
rmse_gbm=sqrt(mean((test$Claims - pred_gbm)^2))

cat("GBM on test set:\n")
```

```
## GBM on test set:
```

```
cat("MAE:", mae_gbm, "\n")
```

```
## MAE: 16.61696
```

```
cat("RMSE:", rmse_gbm, "\n")
```

```
## RMSE: 26.5938
```

```
dev_gbm=sum(2 * (test$Claims * log(pmax(test$Claims,1)/pred_gbm) - (test$Claims - pred_gbm)))  
cat("Deviance:", dev_gbm, "\n")
```

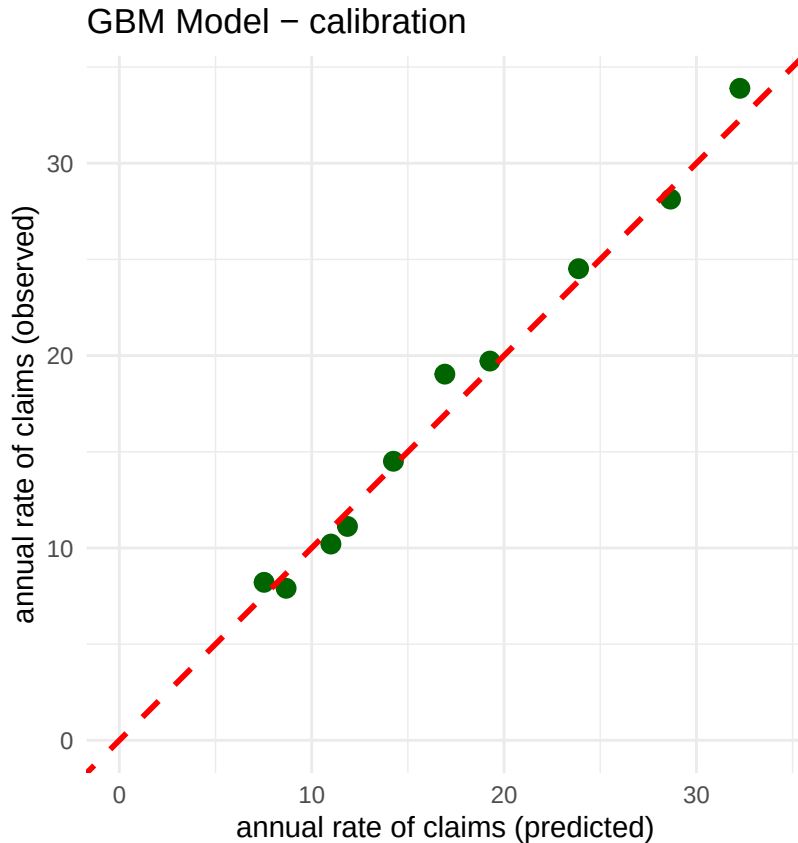
```
## Deviance: 1085657
```

```
calib_df_gbm=test %>%  
  mutate(pred = pred_gbm,  
         decile = ntile(pred, 10))
```

```
calibration_gbm=calib_df_gbm %>%  
  group_by(decile) %>%  
  summarise(  
    observed = sum(Claims),  
    expected = sum(pred),  
    exposure = sum(Exposure),  
    .groups = "drop"  
  ) %>%  
  mutate(  
    obs_rate = observed / exposure,  
    exp_rate = expected / exposure  
  )
```

```
# Plot per GBM
```

```
ggplot(calibration_gbm, aes(x = exp_rate, y = obs_rate)) +  
  geom_point(size = 3, color = "darkgreen") +  
  geom_abline(linetype = "dashed", color = "red", linewidth = 1) +  
  labs(  
    title = "GBM Model - calibration",  
    x = "annual rate of claims (predicted)",  
    y = "annual rate of claims (observed)"  
  ) +  
  theme_minimal() +  
  coord_fixed() +  
  xlim(0, max(calibration_gbm$obs_rate, calibration_gbm$exp_rate)) +  
  ylim(0, max(calibration_gbm$obs_rate, calibration_gbm$exp_rate))
```



Now that we constructed the two models for the frequency, we can put in comparison the metrics of both. Below there is a Model Comparison Table:

Metrics	Negative Binomial	GBM
MAE	16.1646	16.61696
RMSE	26.37752	26.5938
DEVIANCE	1024154	1085657

The Negative Binomial has better performances on the three metrics that we studied. So, for the frequency, the linear model with Negative Binomial Distribution is preferred. This can be address to the fact that the GBM can give an overfitted model.

```
library(tidyverse)
library(readxl)
library(gbm)
```

Modelling the claim severity

We now define the utility functions that we will use for the severity models, together with the path of the dataset. Since this file can also be appended later to the main assignment, we look for the Excel file in both possible locations.

```

options(stringsAsFactors = FALSE)

mae=function(y, p) mean(abs(y - p), na.rm = TRUE)
rmse=function(y, p) sqrt(mean((y - p)^2, na.rm = TRUE))

calibration_table=function(actual, pred, model_name, n_groups = 10) {
  data.frame(actual = actual, pred = pred) %>%
    mutate(decile = ntile(pred, n_groups)) %>%
    group_by(decile) %>%
    summarise(
      n = n(),
      avg_actual = mean(actual, na.rm = TRUE),
      avg_pred = mean(pred, na.rm = TRUE),
      .groups = "drop"
    ) %>%
    mutate(model = model_name) %>%
    select(model, everything())
}

align_factor_levels=function(train_df, test_df, columns) {
  for (col_nm in columns) {
    if (is.factor(train_df[[col_nm]])) {
      train_df[[col_nm]] <- droplevels(train_df[[col_nm]])
      test_df[[col_nm]] <- factor(test_df[[col_nm]], levels = levels(train_df[[col_nm]]))
    }
  }
}

list(train = train_df, test = test_df)
}

fit_gbm_model=function(train_df, predictors, target_name) {
  train_log=train_df
  train_log$log_target=log(train_log[[target_name]])

  gbm_formula=as.formula(
    paste("log_target ~", paste(predictors, collapse = " + "))
  )

  model=gbm(
    formula = gbm_formula,
    data = train_log,
    distribution = "gaussian",
    n.trees = 3000,
    interaction.depth = 3,
    shrinkage = 0.01,
    n.minobsinnode = 100,
    bag.fraction = 0.7,
    cv.folds = 5,
    train.fraction = 1.0,
    verbose = FALSE
  )

  best_iter=gbm.perf(model, method = "cv", plot.it = FALSE)
}

```



```

train_pred_log=predict(model, newdata = train_log, n.trees = best_iter, type = "response")
smearing_factor=mean(exp(train_log$log_target - train_pred_log), na.rm = TRUE)

list(
  model = model,
  best_iter = best_iter,
  distribution_used = "gaussian_on_log_severity",
  smearing_factor = smearing_factor,
  prediction_fun = function(new_df) {
    pred_log <- predict(model, newdata = new_df, n.trees = best_iter, type = "response")
    pmax(exp(pred_log) * smearing_factor, .Machine$double.eps)
  }
)
}

data_candidates=c(
  file.path("insuranceportfolio.xlsx"),
  file.path("insuranceportfolio.xlsx")
)

DATA_FILE=data_candidates[file.exists(data_candidates)][1]

if (is.na(DATA_FILE)) {
  stop("The Excel dataset was not found.")
}

```

Now we focus on modelling the **severity** of claims.
In this case the target variable is:

$$\text{severity} = \frac{\text{cost_claims_year}}{\text{n_medical_services}}$$

So, differently from the frequency model, here we are not modelling the number of events but the average cost per service. Since severity is defined only when there is at least one claim, we keep only the observations with positive claims and positive severity.

We now import the data and prepare the variables:

```

df=read_excel(DATA_FILE)
df=as.data.frame(df)

df=df %>%
  mutate(
    lapse = as.factor(lapse),
    type_policy = as.factor(type_policy),
    type_policy_dg = as.factor(type_policy_dg),
    type_product = as.factor(type_product),
    reimbursement = as.factor(reimbursement),
    new_business = as.factor(new_business),
    distribution_channel = as.factor(distribution_channel),
    gender = as.factor(gender),
    C_H = as.factor(C_H),
    C_GI = as.factor(C_GI),
    C_C = as.factor(C_C),

```

```

    severity = ifelse(n_medical_services > 0, cost_claims_year / n_medical_services, NA_real_)
  )

df_sev=df %>%
  filter(severity > 0, n_medical_services > 0)

```

We use a time-based split because it is more realistic for insurance applications: we train the models on 2017 and 2018, then we evaluate them on 2019.

```

train=df_sev %>%
  filter(period %in% c(2017, 2018))

test=df_sev %>%
  filter(period == 2019)

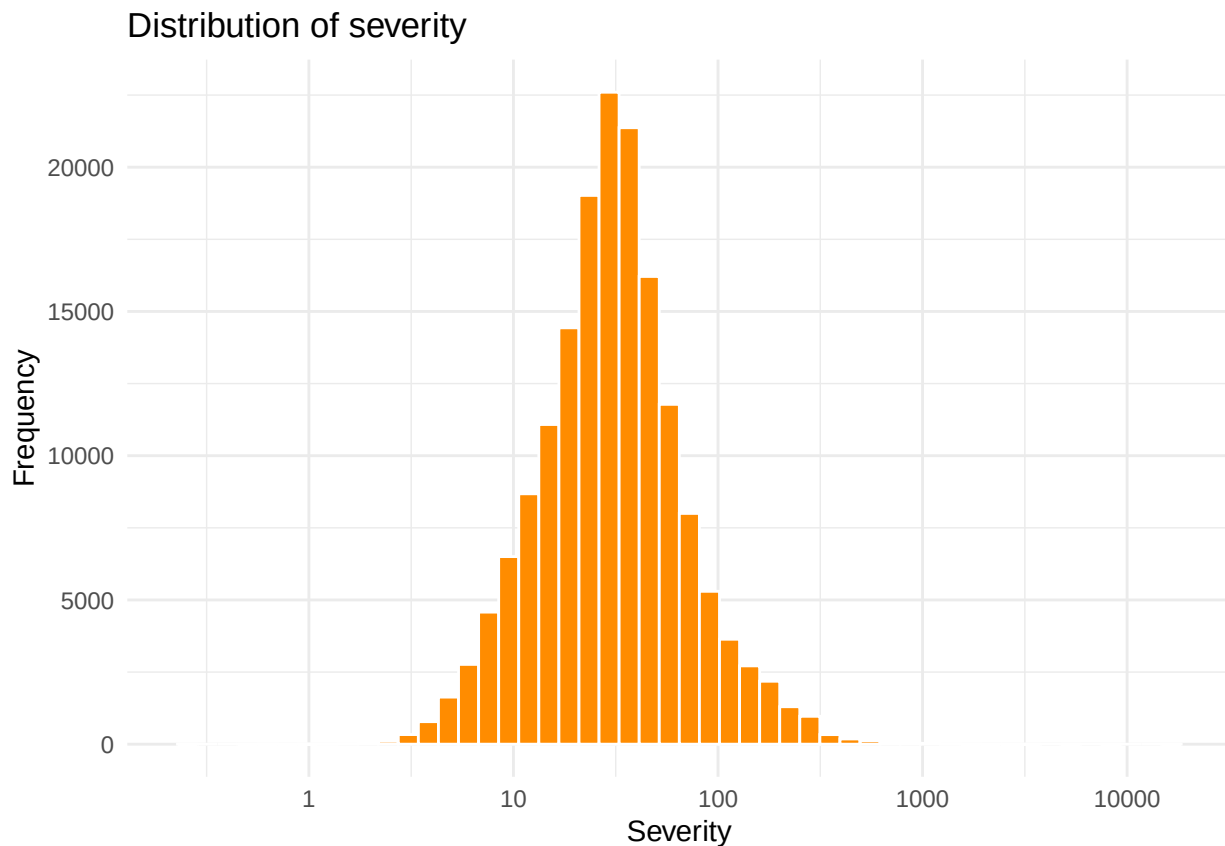
```

We first plot the distribution of severity:

```

ggplot(df_sev, aes(x = severity)) +
  geom_histogram(bins = 50, fill = "darkorange", color = "white") +
  scale_x_log10() +
  theme_minimal() +
  labs(title = "Distribution of severity", x = "Severity", y = "Frequency")

```



The distribution is strictly positive and strongly right-skewed, so two natural candidates are a **Gamma GLM with log link** and a **GBM on log(severity)**.

We now choose the predictors exactly as in the pipeline and we remove the variables that are all missing or constant in the training set.

```
preferred_predictors=c(
  "age",
  "gender",
  "type_policy",
  "type_policy_dg",
  "type_product",
  "premium",
  "reimbursement",
  "new_business",
  "distribution_channel",
  "C_H",
  "C_GI",
  "C_C"
)

predictors=intersect(preferred_predictors, names(df_sev))

all_na_cols=predictors[vapply(train[predictors], function(x) all(is.na(x)), logical(1))]
predictors=setdiff(predictors, all_na_cols)

constant_cols=predictors[vapply(train[predictors], function(x) {
  length(unique(stats::na.omit(x))) <= 1
}, logical(1))]
predictors=setdiff(predictors, constant_cols)

aligned=align_factor_levels(train, test, predictors)
train=aligned$train
test=aligned$test

model_columns=c("severity", predictors)
train_model=train[complete.cases(train[, model_columns, drop = FALSE]), model_columns, drop = FALSE]
test_model=test[complete.cases(test[, model_columns, drop = FALSE]), model_columns, drop = FALSE]

print(nrow(df))

## [1] 228711

print(nrow(df_sev))

## [1] 166877

print(nrow(train_model))

## [1] 103125

print(nrow(test_model))

## [1] 53448
```

```
print(predictors)
```

```
## [1] "age"          "gender"        "type_policy"
## [4] "type_policy_dg" "type_product"  "premium"
## [7] "reimbursement" "new_business"  "distribution_channel"
## [10] "C_H"          "C_GI"          "C_C"
```

The pipeline keeps the same covariates for both models, so the final comparison is fair. In addition, by aligning the factor levels between train and test we avoid errors on unseen categories.

Gamma GLM

The first model is a **Gamma GLM with log link**. This is a standard actuarial choice for positive and right-skewed claim severities.

In this case we model:

$$\log(E[\text{Severity}]) = \beta_0 + \beta_1 x_1 + \dots$$

So we do not need an exposure offset here, because the target is already the average cost per service.

```
gamma_formula=as.formula(
  paste("severity ~", paste(predictors, collapse = " + "))
)
```

```
gamma_glm=glm(
  formula = gamma_formula,
  data = train_model,
  family = Gamma(link = "log")
)
```

```
summary(gamma_glm)
```

```
##
## Call:
## glm(formula = gamma_formula, family = Gamma(link = "log"), data = train_model)
##
## Coefficients: (1 not defined because of singularities)
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   3.913e+00  4.773e-01   8.197 2.50e-16 ***
## age          -4.180e-04  1.914e-04  -2.184 0.028990 *
## genderM       8.896e-02  7.218e-03  12.324 < 2e-16 ***
## type_policyI   7.504e-02  1.416e-02   5.298 1.17e-07 ***
## type_policy_dgC2 -3.739e-03  2.826e-02  -0.132 0.894753
## type_policy_dgC3  4.799e-02  1.556e-02   3.084 0.002040 **
## type_policy_dgC4  3.523e-01  7.036e-02   5.007 5.54e-07 ***
## type_policy_dgI  -9.962e-02  1.130e-02  -8.816 < 2e-16 ***
## type_policy_dgSE      NA         NA      NA      NA
## type_productI    1.651e-01  4.698e-01   0.351 0.725281
## type_productP   -1.274e+00  4.935e-01  -2.582 0.009826 **
## type_productS    1.998e-01  4.681e-01   0.427 0.669447
```

## premium	1.492e-04	8.091e-06	18.441	< 2e-16	***
## reimbursementYes	1.560e+00	1.560e-01	9.997	< 2e-16	***
## new_businessYes	-7.145e-03	1.400e-02	-0.510	0.609779	
## distribution_channelD	4.459e-02	1.204e-01	0.370	0.711176	
## distribution_channelI	2.258e-02	8.620e-03	2.619	0.008817	**
## C_HH2	1.626e-01	3.691e-02	4.405	1.06e-05	***
## C_HH3	8.884e-02	3.687e-02	2.409	0.015985	*
## C_HH4	7.338e-02	3.814e-02	1.924	0.054370	.
## C_HH5	-2.209e-02	3.813e-02	-0.579	0.562443	
## C_HH6	1.521e-02	3.598e-02	0.423	0.672498	
## C_GI2	-1.807e-01	8.888e-02	-2.033	0.042056	*
## C_GI3	-1.476e-01	9.915e-02	-1.489	0.136501	
## C_GI4	-6.412e-01	2.588e-01	-2.477	0.013250	*
## C_GI5	-5.404e-01	1.217e-01	-4.441	8.95e-06	***
## C_GI6	-4.570e-01	1.112e-01	-4.110	3.96e-05	***
## C_GI7	-1.239e+00	2.443e-01	-5.072	3.93e-07	***
## C_GI8	-4.813e-01	1.702e-01	-2.827	0.004700	**
## C_GI9	-5.909e-01	1.533e-01	-3.853	0.000117	***
## C_GI10	-8.154e-01	2.448e-01	-3.330	0.000867	***
## C_GI11	-6.221e-01	1.334e-01	-4.664	3.10e-06	***
## C_GI12	-5.964e-01	2.166e-01	-2.754	0.005894	**
## C_GI13	-8.120e-01	1.314e-01	-6.180	6.43e-10	***
## C_GI14	-7.255e-01	1.508e-01	-4.811	1.50e-06	***
## C_GI15	-6.012e-01	1.693e-01	-3.551	0.000384	***
## C_GI16	-5.037e-01	1.502e-01	-3.355	0.000795	***
## C_GI17	-6.793e-01	1.311e-01	-5.184	2.18e-07	***
## C_GI18	-7.889e-01	1.238e-01	-6.373	1.87e-10	***
## C_GI19	-6.911e-01	1.425e-01	-4.851	1.23e-06	***
## C_GI20	-8.372e-01	1.195e-01	-7.003	2.51e-12	***
## C_GI21	-7.904e-01	1.255e-01	-6.298	3.02e-10	***
## C_GI22	-7.139e-01	1.190e-01	-5.998	2.00e-09	***
## C_GI23	-7.264e-01	1.277e-01	-5.687	1.29e-08	***
## C_GI24	-7.998e-01	1.335e-01	-5.991	2.10e-09	***
## C_GI25	-4.324e-01	1.262e-01	-3.426	0.000613	***
## C_GI26	-7.164e-01	1.048e-01	-6.836	8.19e-12	***
## C_GI27	-7.219e-01	1.156e-01	-6.247	4.21e-10	***
## C_GI28	-6.669e-01	1.500e-01	-4.447	8.72e-06	***
## C_GI29	-6.123e-01	1.180e-01	-5.190	2.11e-07	***
## C_GI30	-6.010e-01	1.388e-01	-4.332	1.48e-05	***
## C_GI31	-6.496e-01	1.018e-01	-6.382	1.75e-10	***
## C_GI32	-6.093e-01	1.212e-01	-5.027	4.99e-07	***
## C_GI33	-7.493e-01	1.055e-01	-7.102	1.24e-12	***
## C_GI34	-7.511e-01	1.088e-01	-6.904	5.08e-12	***
## C_GI35	-7.521e-01	9.359e-02	-8.036	9.38e-16	***
## C_GI36	-9.459e-01	1.101e-01	-8.593	< 2e-16	***
## C_GI37	-6.003e-01	1.110e-01	-5.406	6.46e-08	***
## C_GI38	-6.691e-01	9.807e-02	-6.823	8.98e-12	***
## C_GI39	-7.826e-01	1.152e-01	-6.796	1.08e-11	***
## C_GI40	-7.960e-01	1.076e-01	-7.397	1.41e-13	***
## C_GI41	-6.281e-01	9.889e-02	-6.352	2.14e-10	***
## C_GI42	-9.227e-01	1.137e-01	-8.116	4.89e-16	***
## C_GI43	-6.770e-01	1.008e-01	-6.720	1.83e-11	***
## C_GI44	-8.648e-01	1.164e-01	-7.432	1.08e-13	***
## C_GI45	-8.559e-01	1.130e-01	-7.573	3.69e-14	***

## C_GI46	-7.246e-01	9.902e-02	-7.317	2.55e-13	***
## C_GI47	-6.967e-01	9.455e-02	-7.368	1.74e-13	***
## C_GI48	-6.459e-01	1.094e-01	-5.902	3.60e-09	***
## C_GI49	-7.318e-01	1.076e-01	-6.803	1.03e-11	***
## C_GI50	-6.158e-01	1.082e-01	-5.694	1.25e-08	***
## C_GI51	-6.324e-01	1.009e-01	-6.268	3.67e-10	***
## C_GI52	-7.508e-01	1.059e-01	-7.090	1.35e-12	***
## C_GI53	-6.967e-01	1.041e-01	-6.695	2.16e-11	***
## C_GI54	-5.957e-01	1.040e-01	-5.725	1.04e-08	***
## C_GI55	-7.284e-01	1.073e-01	-6.788	1.14e-11	***
## C_GI56	-6.782e-01	9.910e-02	-6.843	7.79e-12	***
## C_GI57	-7.527e-01	1.062e-01	-7.091	1.34e-12	***
## C_GI58	-6.964e-01	9.858e-02	-7.064	1.62e-12	***
## C_GI59	-7.467e-01	9.831e-02	-7.596	3.09e-14	***
## C_GI60	-6.151e-01	9.767e-02	-6.297	3.04e-10	***
## C_GI61	-6.337e-01	9.908e-02	-6.396	1.60e-10	***
## C_GI62	-7.510e-01	9.835e-02	-7.636	2.26e-14	***
## C_GI63	-8.105e-01	9.459e-02	-8.568	< 2e-16	***
## C_GI64	-7.749e-01	9.126e-02	-8.491	< 2e-16	***
## C_GI65	-7.287e-01	9.134e-02	-7.978	1.50e-15	***
## C_GI66	-7.809e-01	1.029e-01	-7.591	3.20e-14	***
## C_GI67	-7.070e-01	1.051e-01	-6.727	1.74e-11	***
## C_GI68	-7.881e-01	9.394e-02	-8.389	< 2e-16	***
## C_GI69	-7.397e-01	1.004e-01	-7.370	1.72e-13	***
## C_GI70	-8.045e-01	9.692e-02	-8.300	< 2e-16	***
## C_GI71	-7.568e-01	9.453e-02	-8.006	1.20e-15	***
## C_GI72	-7.024e-01	9.835e-02	-7.143	9.22e-13	***
## C_GI73	-8.061e-01	9.995e-02	-8.064	7.44e-16	***
## C_GI74	-8.646e-01	9.288e-02	-9.309	< 2e-16	***
## C_GI75	-7.640e-01	9.026e-02	-8.465	< 2e-16	***
## C_GI76	-7.391e-01	9.211e-02	-8.025	1.03e-15	***
## C_GI77	-7.665e-01	9.228e-02	-8.307	< 2e-16	***
## C_GI78	-6.314e-01	8.755e-02	-7.212	5.56e-13	***
## C_GI79	-6.497e-01	9.488e-02	-6.848	7.53e-12	***
## C_GI80	-6.976e-01	9.080e-02	-7.683	1.56e-14	***
## C_GI81	-5.555e-01	8.952e-02	-6.205	5.49e-10	***
## C_GI82	-6.722e-01	9.089e-02	-7.396	1.41e-13	***
## C_GI83	-6.897e-01	9.023e-02	-7.644	2.12e-14	***
## C_GI84	-6.368e-01	8.924e-02	-7.136	9.70e-13	***
## C_GI85	-6.982e-01	8.710e-02	-8.016	1.11e-15	***
## C_GI86	-6.457e-01	8.664e-02	-7.452	9.23e-14	***
## C_GI87	-7.270e-01	8.883e-02	-8.185	2.76e-16	***
## C_GI88	-6.724e-01	8.667e-02	-7.758	8.74e-15	***
## C_GI89	-7.523e-01	8.960e-02	-8.397	< 2e-16	***
## C_GI90	-6.531e-01	8.817e-02	-7.407	1.30e-13	***
## C_GI91	-6.439e-01	8.757e-02	-7.353	1.95e-13	***
## C_GI92	-7.489e-01	8.643e-02	-8.665	< 2e-16	***
## C_GI93	-6.063e-01	8.636e-02	-7.020	2.22e-12	***
## C_GI94	-6.413e-01	8.563e-02	-7.489	6.97e-14	***
## C_GI95	-6.604e-01	8.553e-02	-7.721	1.17e-14	***
## C_GI96	-6.762e-01	8.548e-02	-7.910	2.60e-15	***
## C_GI97	-6.705e-01	8.458e-02	-7.927	2.27e-15	***
## C_GI98	-6.438e-01	8.452e-02	-7.617	2.61e-14	***
## C_GI99	-5.845e-01	8.443e-02	-6.923	4.45e-12	***

```
## C_GI100          -5.565e-01  8.416e-02  -6.613  3.78e-11 ***
## C_CC2            -5.109e-02  2.494e-02  -2.048  0.040532 *
## C_CC3            -8.324e-02  1.467e-02  -5.675  1.39e-08 ***
## C_CC4            6.925e-01  1.786e-02  38.763  < 2e-16 ***
## C_CC5            9.620e-02  2.512e-02   3.830  0.000128 ***
## C_CC6            5.535e-02  1.401e-02   3.950  7.83e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for Gamma family taken to be 1.313086)
##
## Null deviance: 68436  on 103124  degrees of freedom
## Residual deviance: 56624  on 103000  degrees of freedom
## AIC: 957735
##
## Number of Fisher Scoring iterations: 7
```

We now evaluate the Gamma GLM on the test set:

```
pred_gamma_glm=predict(gamma_glm, newdata = test_model, type = "response")
actual=test_model$severity

mae_gamma=mae(actual, pred_gamma_glm)
rmse_gamma=rmse(actual, pred_gamma_glm)

cat("Gamma GLM on test set:\n")
```

```
## Gamma GLM on test set:
```

```
cat("MAE:", mae_gamma, "\n")
```

```
## MAE: 28.40825
```

```
cat("RMSE:", rmse_gamma, "\n")
```

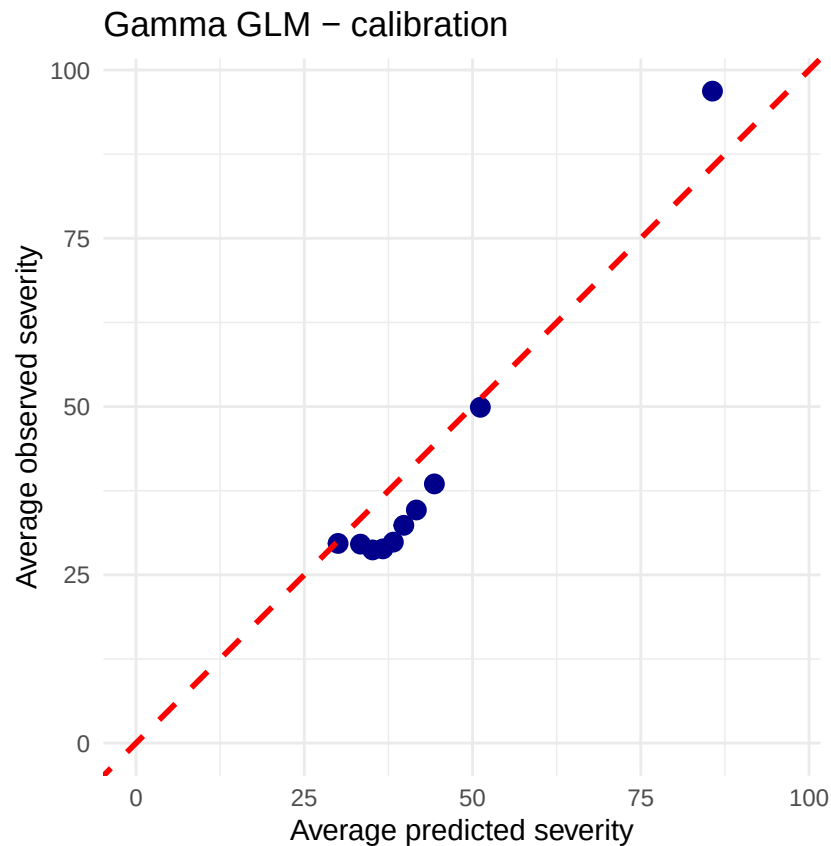
```
## RMSE: 59.61566
```

```
calibration_table = function(actual, pred, model_name, n_groups = 10) {
  data.frame(actual = actual, pred = pred) %>%
    mutate(decile = ntile(pred, n_groups)) %>%
    group_by(decile) %>%
    summarise(
      n = n(),
      avg_actual = mean(actual, na.rm = TRUE),
      avg_pred = mean(pred, na.rm = TRUE),
      .groups = "drop"
    ) %>%
    mutate(model = model_name) %>%
    dplyr::select(model, everything()) # <-- ECCO LA CORREZIONE
}
```

```

calibration_gamma=calibration_table(actual, pred_gamma_glm, "gamma_glm")
max_val_gamma = max(c(calibration_gamma$avg_actual, calibration_gamma$avg_pred), na.rm = TRUE)
ggplot(calibration_gamma, aes(x = avg_pred, y = avg_actual)) +
  geom_point(size = 3, color = "darkblue") +
  geom_abline(linetype = "dashed", color = "red", linewidth = 1) +
  labs(
    title = "Gamma GLM - calibration",
    x = "Average predicted severity",
    y = "Average observed severity"
  ) +
  theme_minimal() +
  coord_fixed(xlim = c(0, max_val_gamma), ylim = c(0, max_val_gamma))

```



In the current run the Gamma GLM gives a MAE of about 28.41 and a RMSE of about 59.62. The calibration plot shows that the model follows the general trend quite well, but in the highest decile it still underestimates the most severe claims.

Gradient Boosting Machine (GBM)

The second model is a **Gradient Boosting Machine**. As in the frequency section, this model is more flexible because it is non-parametric and can capture non-linear effects and interactions.

Here we do not model the severity directly, but the logarithm of severity:

$$\log(\text{Severity}) = F(x_1, x_2, \dots)$$

Then we transform the predictions back to the original scale using a smearing correction. This is exactly the idea used in the standalone pipeline.

```
set.seed(42)
gbm_fit=fit_gbm_model(train_model, predictors, "severity")
best_iter=gbm_fit$best_iter

cat("Best number of trees:", best_iter, "\n")
```

```
## Best number of trees: 3000
```

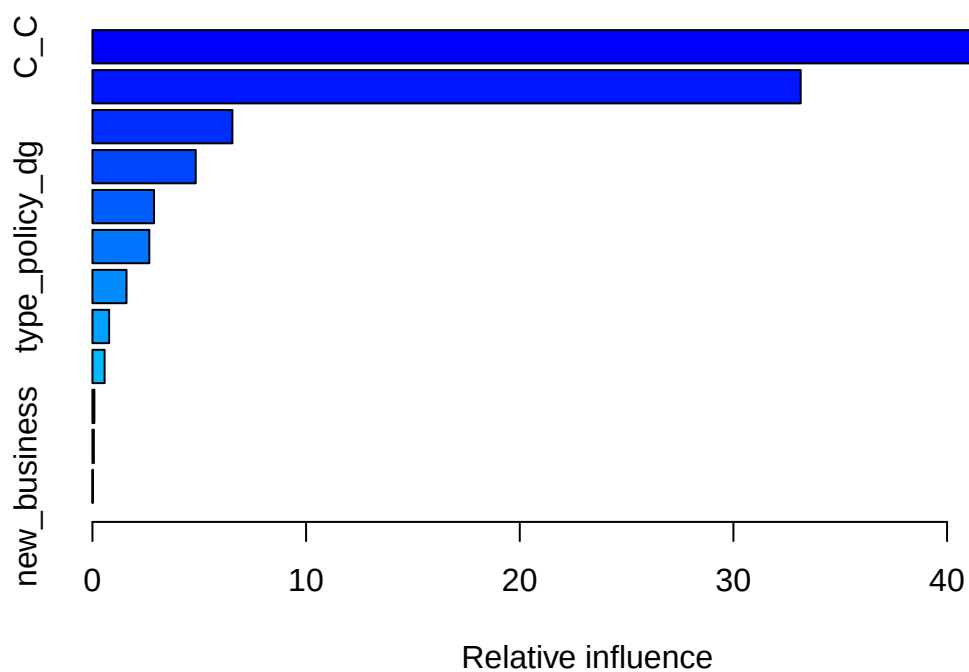
```
cat("Distribution used:", gbm_fit$distribution_used, "\n")
```

```
## Distribution used: gaussian_on_log_severity
```

```
cat("Smearing factor:", gbm_fit$smearing_factor, "\n")
```

```
## Smearing factor: 1.307992
```

```
summary(gbm_fit$model, n.trees = best_iter)
```



```
##          var      rel.inf
## C_C      46.79894346
```

```
## C_GI                C_GI 33.14712598
## premium             premium 6.55138529
## C_H                 C_H 4.83361630
## age                 age 2.88221980
## type_policy_dg      type_policy_dg 2.66384963
## type_product        type_product 1.59329911
## gender              gender 0.78230859
## reimbursement      reimbursement 0.57019056
## distribution_channel distribution_channel 0.09145384
## type_policy         type_policy 0.06787953
## new_business        new_business 0.01772788
```

The variable importance plot shows that the geographical variables are the most relevant ones. In particular C_C and C_GI explain a large part of the severity variation, while variables like `new_business` and `type_policy` have a much smaller impact.

We now evaluate the GBM on the test set:

```
pred_gbm_lognormal=gbm_fit$prediction_fun(test_model)

mae_gbm=mae(actual, pred_gbm_lognormal)
rmse_gbm=rmse(actual, pred_gbm_lognormal)

cat("GBM on test set:\n")
```

```
## GBM on test set:
```

```
cat("MAE:", mae_gbm, "\n")
```

```
## MAE: 27.93219
```

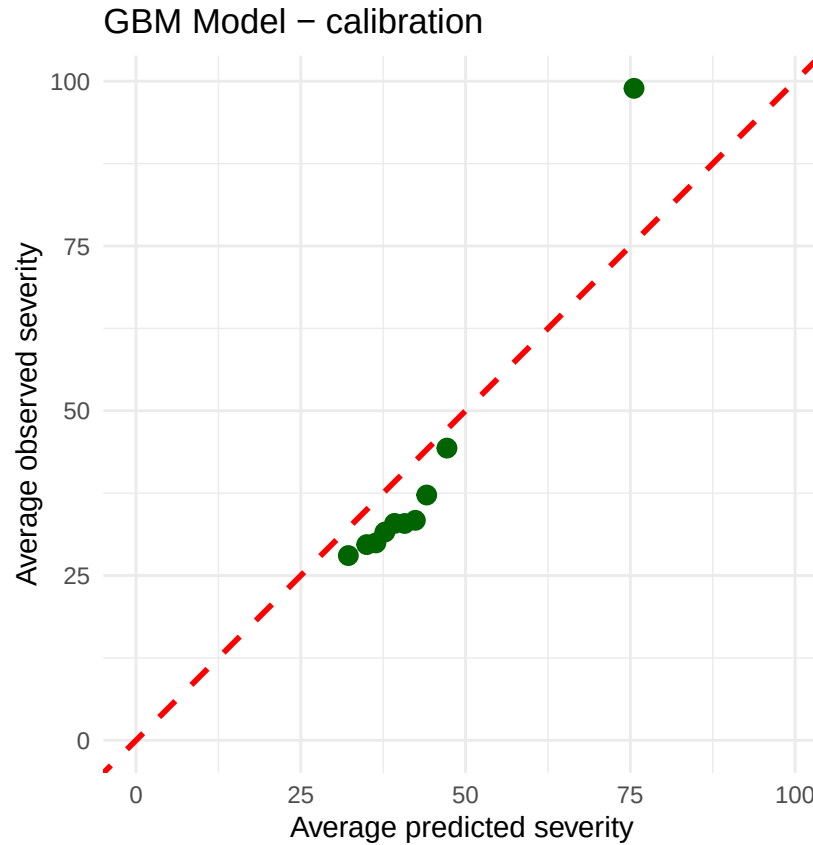
```
cat("RMSE:", rmse_gbm, "\n")
```

```
## RMSE: 59.19906
```

```
calibration_gbm=calibration_table(actual, pred_gbm_lognormal, "gbm_lognormal")

max_val_gbm = max(c(calibration_gbm$avg_actual, calibration_gbm$avg_pred), na.rm = TRUE)

ggplot(calibration_gbm, aes(x = avg_pred, y = avg_actual)) +
  geom_point(size = 3, color = "darkgreen") +
  geom_abline(linetype = "dashed", color = "red", linewidth = 1) +
  labs(
    title = "GBM Model - calibration",
    x = "Average predicted severity",
    y = "Average observed severity"
  ) +
  theme_minimal() +
  coord_fixed(xlim = c(0, max_val_gbm), ylim = c(0, max_val_gbm))
```



In the current run the GBM gives a MAE of about 27.93 and a RMSE of about 59.20. So the improvement compared to the Gamma GLM is not extremely large, but it is still slightly better on both metrics.

Final comparison

Now that we constructed the two models for the severity, we can put in comparison the metrics of both. Below there is a Model Comparison Table:

Metrics	Gamma GLM	GBM
MAE	28.4083	27.9322
RMSE	59.6157	59.1991

We can also look at the comparison directly from R:

```
model_comparison=data.frame(
  model = c("gamma_glm", "gbm_lognormal"),
  MAE = c(mae_gamma, mae_gbm),
  RMSE = c(rmse_gamma, rmse_gbm)
)

print(model_comparison)
```

```
##           model      MAE      RMSE
```

```
## 1      gamma_glm 28.40825 59.61566
## 2 gbm_lognormal 27.93219 59.19906
```

The GBM has slightly better performances on the two metrics that we studied, so for the severity analysis the tree-based model is marginally preferred. At the same time the Gamma GLM remains a very solid benchmark, because it is simpler to interpret and its predictive performance is very close to the GBM one.

Risk Profiles and Pure Premium

Risk Profile

We now define the **risk profiles**, in order to avoid factor-level errors with the severity model, we consider a base row from our test set and modify the key variable to define the risk levels.

```
base_row=test_model[1, ]
risk_profiles=rbind(base_row, base_row, base_row)
```

Based on what we discovered during our exploratory data analysis we can choose the following profiles:

- **low risk**: Age 30, Female, C_C = C4
- **medium risk**: Age 45, Male, C_C = C1
- **high risk**: Age 75, Male, C_C = C2

```
risk_profiles$Profile=c("Low Risk", "Medium Risk", "High Risk")
risk_profiles$Age=c(30, 45, 75)
risk_profiles$gender=factor(c("F", "M", "M"), levels = levels(test_model$gender))
risk_profiles$C_C=factor(c("C4", "C1", "C2"), levels = levels(test_model$C_C))
```

We standardize the exposure to 1:

```
risk_profiles$Exposure=c(1, 1, 1)
risk_profiles$logExposure=log(risk_profiles$Exposure)
```

We now predict the frequency considering the best model (**negative binomial**) and the worst one (**GBM**):

```
risk_profiles$FreqNB=predict(glm_nb_geo, newdata = risk_profiles, type = "response")
risk_profiles$FreqGBM=predict(gbm_train, newdata = risk_profiles, n.trees = best.iter, type = "response")
```

We now predict the severity considering the best model (**GBM**) and the worst one (**gamma GLM**):

```
risk_profiles$SevGBM=gbm_fit$prediction_fun(risk_profiles)
risk_profiles$SevGamma=predict(gamma_glm, newdata = risk_profiles, type = "response")
```

Pure Premium

We finally calculate the **Pure Premium**:

$$\text{Pure premium} = \text{Expected Frequency} * \text{Expected Severity}$$

```
risk_profiles$PurePremiumBest=risk_profiles$FreqNB * risk_profiles$SevGBM
risk_profiles$PurePremiumWorst=risk_profiles$FreqGBM * risk_profiles$SevGamma
```

We now show the results:

```
library(knitr)
results_table=risk_profiles %>%
  dplyr::select(Profile, FreqNB, FreqGBM, SevGBM, SevGamma, PurePremiumBest, PurePremiumWorst) %>%
  mutate(across(where(is.numeric), ~ round(., 2)))

kable(results_table, caption = "Expected Claims, Severity, and Pure Premium by Risk Profile")
```

Table 3: Expected Claims, Severity, and Pure Premium by Risk Profile

Profile	FreqNB	FreqGBM	SevGBM	SevGamma	PurePremiumBest	PurePremiumWorst
Low Risk	8.71	11.57	80.20	84.51	698.36	977.70
Medium Risk	13.22	10.63	47.59	45.93	629.28	488.08
High Risk	32.86	35.87	40.63	43.10	1335.07	1545.87

##Actual vs Expected Loss Analysis

```
library(kableExtra)

portfolio_test=ataclean %>%
  filter(period == 2019,
         !is.na(exposure_time), exposure_time > 0,
         !is.na(age), !is.na(gender), !is.na(type_product), !is.na(C_C)) %>%
  mutate(
    Claims = n_medical_services,
    Exposure = exposure_time,
    logExposure = log(exposure_time),
    Observed_Loss = ifelse(is.na(cost_claims_year), 0, cost_claims_year)
  )
aligned_portfolio=align_factor_levels(train_model, portfolio_test, predictors)
portfolio_test=aligned_portfolio$test
portfolio_test$Pred_Freq_GLM=predict(glm_nb_geo, newdata = portfolio_test, type = "response")
portfolio_test$Pred_Freq_GBM=predict(gbm_train, newdata = portfolio_test, n.trees = best.iter, type = "response")
portfolio_test$Pred_Sev_GLM=predict(gamma_glm, newdata = portfolio_test, type = "response")
portfolio_test$Pred_Sev_GBM= gbm_fit$prediction_fun(portfolio_test)
portfolio_test <- portfolio_test %>%
  mutate(
    Pred_Loss_GLM = Pred_Freq_GLM * Pred_Sev_GLM,
    Pred_Loss_GBM = Pred_Freq_GBM * Pred_Sev_GBM
  )
portfolio_comparison=portfolio_test %>%
  group_by(C_C) %>%
  summarise(
    Observed = sum(Observed_Loss, na.rm = TRUE),
    GLM = sum(Pred_Loss_GLM, na.rm = TRUE),
    GBM = sum(Pred_Loss_GBM, na.rm = TRUE),
```

```

    .groups = "drop"
  )
  portfolio_ratios=portfolio_comparison %>%
    mutate(
      C_C = C_C,
      Observed_Ratio = "-",
      GLM = GLM / Observed,
      GBM = GBM / Observed
    ) %>%
    dplyr::select(C_C, Observed_Ratio, GLM, GBM)
  loss_table=portfolio_comparison %>%
    mutate(
      C_C = as.character(C_C),
      Observed = formatC(Observed, format = "f", digits = 0, big.mark = " "),
      GLM = formatC(GLM, format = "f", digits = 0, big.mark = " "),
      GBM = formatC(GBM, format = "f", digits = 0, big.mark = " ")
    )
  colnames(loss_table)[1] ="Geographic Area (C_C)"
  ratio_table=portfolio_ratios %>%
    mutate(
      C_C = as.character(C_C),
      GLM = formatC(GLM, format = "f", digits = 2),
      GBM = formatC(GBM, format = "f", digits = 2)
    )
  colnames(ratio_table)[1:2]=c("Geographic Area (C_C)", "Observed")
  cat("**Observed and predicted losses**\n")

```

```
## **Observed and predicted losses**
```

```
kable(loss_table, align = c("l", "r", "r", "r"))
```

Geographic Area (C_C)	Observed	GLM	GBM
C1	4 701 304	5 244 974	5 015 256
C2	1 154 656	1 793 260	1 687 559
C3	18 623 471	23 001 233	23 404 922
C4	5 169 748	6 518 437	6 215 604
C5	1 222 700	1 559 998	1 492 948
C6	10 996 538	13 686 630	12 841 032

```
cat("\n**Ratio of predicted losses over observed losses**\n")
```

```
##
```

```
## **Ratio of predicted losses over observed losses**
```

```
kable(ratio_table, align = c("l", "r", "r", "r"))
```

Geographic Area (C_C)	Observed	GLM	GBM
C1	-	1.12	1.07

Geographic Area (C_C)	Observed	GLM	GBM
C2	-	1.55	1.46
C3	-	1.24	1.26
C4	-	1.26	1.20
C5	-	1.28	1.22
C6	-	1.24	1.17

Portfolio Validation: Actual vs. Expected Loss Analysis

We performed an Actual vs. Expected loss analysis aggregated by geographic area (C_C) for the out of sample year 2019. The expected losses are derived by multiplying the predicted frequency from the frequency models (Negative Binomial GLM and Poisson GBM) by the predicted severity from the severity models (Gamma GLM and Log-Normal GBM) expecting to get a value as much as possible close to 1.

1. **Systematic Overestimation and Solvency Considerations:** The ratio of predicted over observed losses is always greater than 1.00, ranging from 1.07 to 1.55. This means that both modeling frameworks lean toward overestimating the total financial damage for the year 2019. While a ratio above 1.00 guarantees insurer solvency, ensuring that the collected pure premiums cover the actual claims. It implies a commercial risk of overpricing, which could reduce competitiveness in highly populated regions.
2. **The Impact of the Time-Based Split:** The systematic overestimation shows the realism of our time-based validation approach. Since the models were trained on data from 2017 and 2018, the discrepancy in 2019 strongly suggests a macro-environmental shift, such as a lower baseline of claim occurrences or milder weather conditions reducing medical service utilization. A random train test split would have masked this temporal dependency, this prove that a time-based validation is fundamental for a precise study.
3. **Model Comparison (GLM vs. GBM):** When comparing the two models, the GBM outperforms the traditional GLM framework by delivering ratios closer to 1.00 in almost all clusters. This is an evidence that the non-linear interactions and tree based structures captured by the GBM allow for better out of sample generalization at the portfolio level compared to the constraints of the GLM.

Conclusions

Frequency: relevance of non-linear models

- **Expected Trend** (Negative Binomial): it shows a linear progression, indeed the low-risk profile has fewer claims (8.71), the medium-risk has more (13.22) and the high-risk senior has the highest number (32.86).
- **GBM:** the GBM estimates a higher frequency for the theoretically low risk profile (11.57) compared to the medium one (10.63).

Why did this happen?

As seen in the exploratory data analysis , young women show a frequency peak due to *maternity* or *preventive care*. Since the GBM is a non linear tree based model it was able to capture this interaction between age and gender while the linear model flattened it.

Severity: inverse correlation with frequency

- for both GBM and Gamma GLM severity follows an **inverse trend** relative to frequency.
- the **low risk** profile *costs more* per individual event (80.20-84.51 Euro) compared to the **high risk** senior (40.63-43.10 Euro).

What can we conclude under a Business approach?

This is common in health insurance. Indeed young individuals visit the doctor rarely compared to elders, but when they do it is usually for something expensive or dangerous like surgeries, maternity or life-threatening diseases or injuries. Conversely, high-risk seniors have an high frequency because they need constant check-ups or prescriptions for chronic conditions, but the average cost per visit is lower.

Pure Premium Analysis

The Pure Premium calculates the **total expected cost** by multiplying (Frequency*Severity).

- **High Risk:** the oldest ones are the most expensive profile to insure, with a pure premium exceeding 1,300-1,500 Euros.

- **Low vs Medium Risk:** it seems weird but the low risk profile has an higher premium than the medium risk.

Why does this happens? Which are the possible reasons?

This is a proof that the initial labels low and medium were a priori assumptions based on age/gender, but the model reveals a different behavior. For this reason sometimes defining the profiles in this way can be extremely dangerous.